

# 5 Interconnect

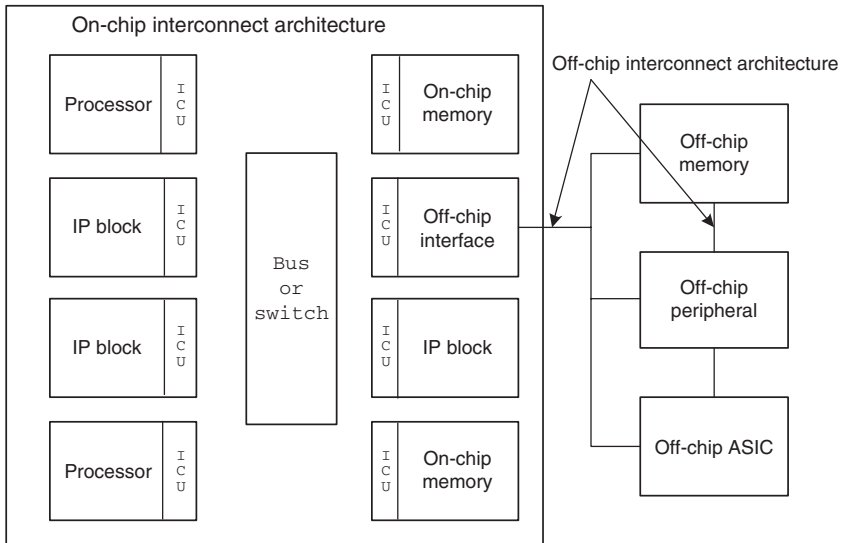
## 5.1 INTRODUCTION

SOC designs usually involve the integration of intellectual property (IP) cores, each separately designed and verified. System integrators can maximize the reuse of design to reduce costs and to lower risks. Frequently the most important issue confronting an SOC integrator is the method by which the IP cores are connected together.

SOC interconnect alternatives extend well beyond conventional computer buses. We first provide an overview of SOC interconnect architectures: bus and network-on-chip (NOC). Bus architectures developed specifically for SOC designs are described and compared. There are many switch-based alternatives to bus-based interconnects. We will not consider ad hoc or fully customized switching interconnects that are not intended for use with a variety of IP cores. Switch-based interconnects as used in SOC interconnects are referred to as NOC technology.

An NOC usually includes an interface level of abstraction, hiding the underlying physical interconnects from the designer. We follow current SOC usage and refer to interconnect as a bus or as an NOC implemented by a switch. In the NOC the switch can be a crossbar, a directly linked interconnect, or a multistage switching network.

There is a great deal of bus and computer interconnect literature. The units being connected are sometimes referred to as agents (in buses) or nodes (in the general interconnect literature); we simply use the term *units*. Since current SOC interconnects usually involve a modest number of units, the chapter provides a simplified view of the interconnect alternatives. A comprehensive treatment of on-chip communication architectures is available elsewhere [193]. For a general discussion of computer interconnection networks, see any of several standard texts [72, 78].



**Figure 5.1** A simplified block diagram of an SOC module in a system context.

## 5.2 OVERVIEW: INTERCONNECT ARCHITECTURES

Figure 5.1 depicts a system that includes an SOC module. The SOC module typically contains a number of IP blocks, one or more of which are processors. In addition, there are various types of on-chip memory serving as cache, data, or instruction storage. Other IP blocks serving application-specific functions, such as graphics processors, video codecs, and network control units, are integrated in the SOC.

The IP blocks in the SOC module need to communicate with each other. They do this through the interconnect, which is accessed through an interconnect interface unit (ICU). The ICU enables a common interface protocol for all SOC modules.

External to the SOC module are off-chip memories, off-chip peripheral devices, and mass storage devices. The cost and performance of the system, therefore, depends on both on-chip and off-chip interconnect structures.

Choosing a suitable interconnect architecture requires the understanding of a number of system level issues and specifications. These are:

1. *Communication Bandwidth.* The rate of information transfer between a module and the surrounding environment in which it operates. Usually measured in bytes per second, the bandwidth requirement of a module dictates to a large extent the type of interconnection required in order to achieve the overall system throughput specification.

## WHAT IS AN NOC?

As SOC terminology has evolved there seems to be only two interconnect strategies: the bus or the NOC. So what exactly is the NOC? Professor Nurmi (in a presentation reported by Leibson [156]) summarized the NOC characteristics:

1. The NOC is more than a single, shared bus.
2. The NOC provides point-to-point connections between any two hosts attached to the network either by crossbar switches or through node-based switches.
3. The NOC provides high aggregate bandwidth through parallel links.
4. In the NOC, communication is separate from computation.
5. The NOC uses a layered approach to communications, although with few network layers due to complexity and expense.
6. NOCs support pipelining and provide intermediate data buffering between sender and receiver.

In the context of the SOC when the designer finds that bus technology provides insufficient bandwidth or connectivity, the obvious alternative is some sort of switch. Any well-designed switched interconnect will clearly satisfy points 2, 3, 4, and 6. Point 5 is not satisfied by ad hoc switching interconnects, where the processor nodes and switching interconnect are interfaced by common, specialized design. But in the SOC, incorporating various vendor IPs ad hoc interconnects is almost never the case. The designer selects a common communications interface (layer) separate from the processor node.

2. *Communication Latency.* The time delay between a module requesting data and receiving a response to the request. Latency may or may not be important in terms of overall system performance. For example, long latency in a video streaming application usually has little or no effect on the user's experience. Watching a movie that is a couple of seconds later than when it is actually broadcast is of no consequence. In contrast, even small, unanticipated latencies in a two-way mobile communication protocol can make it almost impossible to carry out a conversation.
3. *Master and Slave.* These terms concern whether a unit can initiate or react to communication requests. A master, such as a processor, controls transactions between itself and other modules. A slave, such as memory, responds to requests from the master. An SOC design typically has several masters and numerous slaves.

4. *Concurrency Requirement.* The number of independent simultaneous communication channels operating in parallel. Usually, additional channels improve system bandwidth.
5. *Packet or Bus Transaction.* The size and definition of the information transmitted in a single transaction. For a bus, this consists of an address with control bits (read/write, etc.) and data. The same information in an NOC is referred to as a *packet*. The packet consists of a header (address and control) and data (sometimes called the payload).
6. *ICU.* In an interconnect, this unit manages the interconnect protocol and the physical transaction. It can be simple or complex, including out-of-order transaction buffering and management. If the IP core requires a protocol translation to access the bus, the unit is called a *bus wrapper*. In an NOC, this unit manages the protocol for transport of a packet from the IP core to the switching network. It provides packet buffering and out-of-order transaction transmission.
7. *Multiple Clock Domains.* Different IP modules may operate at different clock and data rates. For example, a video camera captures pixel data at a rate governed by the video standard used, while a processor's clock rate is usually determined by the technology and architectural design. As a result, IP blocks inside an SOC often need to operate at different clock frequencies, creating separate timing regions known as clock domains. Crossing between clock domains can cause deadlock and synchronization problems without careful design.

Given a set of communication specifications, a designer can explore the different bandwidth, latency, concurrency, and clock domain requirements of different interconnect architectures, such as bus and NOC. Some examples of these are given in Table 5.1. Other examples include the Avalon Bus for Altera field-programmable gate arrays (FPGAs) [10], the Wishbone Interconnect for use in open-source cores and platforms [189], and the AXI4-Stream interface protocol for FPGA implementation [74].

Designing the interconnect architecture for an SOC requires careful consideration of many requirements, such as those listed above. The rest of this chapter provides an introduction to two interconnect architectures: the bus and the NOC.

### 5.3 BUS: BASIC ARCHITECTURE

The performance of a computer system is heavily dependent on the characteristics of its interconnect architecture. A poorly designed system bus can throttle the transfer of instructions and data between memory and processor, or between peripheral devices and memory. This communication bottleneck is the focus of attention among many microprocessor and system

**TABLE 5.1 Examples of Interconnect Architectures [167]**

| Technology<br>Company | AMBA<br>ARM | AXI (AMBA 3)<br>ARM     | CoreConnect<br>IBM | Smart Interconnect IP<br>Sonics | Nexus<br>Fulcrum        |
|-----------------------|-------------|-------------------------|--------------------|---------------------------------|-------------------------|
| Core type             | Soft/hard   | Soft/hard               | Soft               | Soft                            | Hard                    |
| Architecture          | Bus         | Unidirectional channels | Bus                | Bus                             | NOC using direct switch |
| Bus width             | 8–1024      | 8–1024                  | 32/64/128          | 16                              | 8–128                   |
| Frequency             | 200 MHz     | 400 MHz*                | 100–400 MHz        | 300 MHz                         | 1 GHz                   |
| Maximum BW (GB/s)     | 3           | 6.4*                    | 2.5–24             | 4.8                             | 72                      |
| Minimum latency (ns)  | 5           | 2.5*                    | 15                 | n/a                             | 2                       |

\*As implemented in the ARM PL330 high-speed controller.  
BW, bandwidth.

**TABLE 5.2 Comparison of Bus Interconnect Architectures [198]**

| Standard                       | Speed (MHz) | Area (rbe*) |
|--------------------------------|-------------|-------------|
| AMBA(implementation dependent) | 166–400     | 175,000     |
| CoreConnect                    | 66/133/183  | 160,000     |

\*rbe = register bit equivalent; estimates are approximate and vary by implementation.

manufacturers who, over the last three decades, have adopted a number of bus standards. These include the popular VME bus and the Intel Multibus-II. For systems on a board and personal computers, the evolution includes the instruction set architecture (ISA) bus, the EISA bus, and the now prevalent PCI and PCI Express buses. All these bus standards are designed to connect together integrated circuits (ICs) on a printed circuit board (PCB) or PCBs in a system-on-board implementation.

While these bus standards have served the computing community well, they are not particularly suited for SOC technology. For example, all such system-level buses are designed to drive a backplane, either in a rack-mounted system or on a computer motherboard. This imposes numerous constraints on the bus architecture. For a start, the number of signals available is generally restricted by the limited pin count on an IC package or the number of pins on the PCB connector. Adding an extra pin on a package or a connector is expensive. Furthermore, the speed at which the bus can operate is often limited by the high capacitive load on each bus signal, the resistance of the contacts on the connector, and the electromagnetic noise produced by such fast-switching signals traveling down a PCB track. Finally, drivers for on-chip buses can be much smaller, saving area and power.

Before describing bus operations and bus structures in detail, we provide, in Table 5.2, a comparison of two different bus interconnect architectures, showing size and speed estimates for a typical bus slave.

### 5.3.1 Arbitration and Protocols

Conceptually, the bus is just wires shared by multiple units. In practice, some logic must be present to provide an orderly use of the bus; otherwise, two units may send signals at the same time, causing conflicts. When a unit has exclusive use of the bus, the unit is said to own the bus. Units can be either potentially master units that can request ownership or slave units that are passive and only respond to requests. A bus master is the unit that initiates communication on a computer bus or input/output (I/O) paths. In an SOC, a bus master is a component within the chip, such as a processor. Other units connected to an on-chip bus, such as I/O devices and memory components, are the “slaves.” The bus master controls the bus paths using specific slave addresses and control signals. Moreover, the bus master also controls the flow of data signals directly between the master and the slaves.

A process called arbitration determines ownership. A simple implementation has a centralized arbitration unit with an input from each potential requesting unit. The arbitration unit then grants bus ownership to one requesting unit, as determined by the bus protocol.

A bus protocol is an agreed set of rules for transmitting information between two or more devices over a bus. The protocol determines the following:

- the type and order of data being sent;
- how the sending device indicates that it has finished sending the information;
- the data compression method used, if any;
- how the receiving device acknowledges successful reception of the information; and
- how arbitration is performed to resolve contention on the bus and in what priority, and the type of error checking to be used.

### 5.3.2 Bus Bridge

A bus bridge is a module that connects together two buses, which are not necessarily of the same type. A typical bridge can serve three functions:

1. If the two buses use different protocols, a bus bridge provides the necessary format and standard conversion.
2. A bridge is inserted between two buses to segment them and keep traffic contained within the segments. This improves concurrency: both buses can operate at the same time.
3. A bridge often contains memory buffers and the associated control circuits that allow write posting. When a master on one bus initiates a data transfer to a slave module on another bus through the bridge, the data is temporarily stored in the buffer, allowing the master to proceed to the next transaction before the data are actually written to the slave. By allowing transactions to complete quickly, a bus bridge can significantly improve system performance.

### 5.3.3 Physical Bus Structure

The nature of the bus transaction depends on the physical bus structure (number of wire paths, cycle time, etc.) and the protocol (especially the arbitration support). Multiple bus users must be arbitrated for access to the bus in any given cycle. Thus, arbitration is part of the bus transaction. Simple arbiters have a request cycle wherein signals from the users are prioritized, followed by the acknowledge cycle selecting the user. More complex arbiters add bus control lines and associated logic so that each user is aware of pending bus status and priority. In such designs no cycles are added to the bus transaction for arbitration.

### 5.3.4 Bus Varieties

Busess may be *unified* or *split* (address and data). In the unified bus the address is initially transmitted in a bus cycle followed by one or more data cycles; the split bus has separate buses for each of these functions.

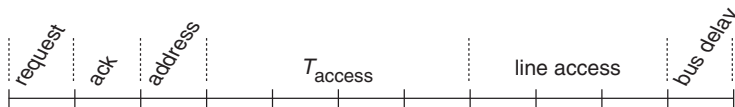
Also, the buses may be single transaction or *tenured*. Tenured buses are occupied by a transaction only during associated addresses or data cycles. Such buses have unit receivers that buffer the messages and create separate address and data transactions.

#### EXAMPLE 5.1 BUS EXAMPLES

There are many possible bus designs with varying combinations of physical bus widths and arbitration protocols. The examples below consider some obvious possibilities. Suppose the bus has a transmission delay of one processor cycle, and the memory (or shared cache) has a four-cycle access delay after an initial address and requires an additional cycle for each sequential data access. The memory is accessed 4 bytes at a time. The data to be transmitted consist of a 16-byte cache line. Address requests are 4 bytes.

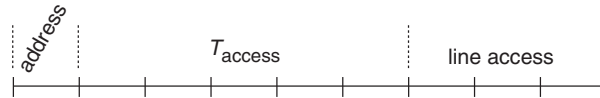
In these examples,  $T_{\text{access}}$  is the time required to access the first word from memory after the address is issued, and line access is the time required to access the remaining words. Also, the last byte of data arrives at the end of the timing template and can be used only after that point.

- (a) *Simple Bus.* This is a single transaction bus with simple request/acknowledge (ack) arbitration. It has a physical width of 4 bytes. The request and ack signals are separate signals but assumed to be part of the bus transaction, so the bus transaction latency is 11 cycles. The first word is sent from memory at the last cycle of  $T_{\text{access}}$ , while the fourth (and last) word is sent from memory at the last cycle of line access. The final bus cycle is to reset the arbiter.

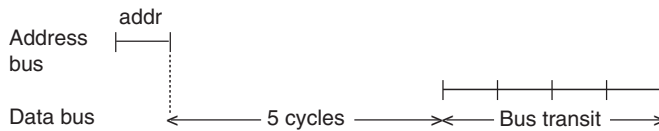


- (b) *Bus with Arbitration Support.* This bus has a more sophisticated arbiter but still has a 4-byte physical width and integrates address and data. There is an additional access cycle (five cycles instead of four) to represent the time to move the address from the bus receiver to the memory. This is not shown in case (a), as simple buses are usually slower with immediate coupling to memory. Now the initial cycles for request and ack are overlapped with bus processing, and the final cycle for resetting the arbiter is not shown in the figure for case (b), so the bus transaction now takes 10 cycles.

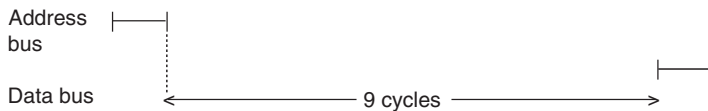




- (c) *Tenured Split Bus, 4 Bytes Wide.* The assumption is that the requested line is fetched into a buffer for five cycles and then transmitted in four cycles. While the transaction latency, including the cycle for the address, is no different from that in case (b) at 10 cycles, the transaction occupies the bus for less than half (four cycles) of that time. The address bus is used for only one cycle out of 10. The remaining time is available for other unrelated transactions to improve communication performance.



- (d) *Tenured Split Bus, 16 Bytes Wide, with a One-Cycle Bus Transaction Time.* As with case (c), the transaction latency is unaffected at 10 cycles. Since the memory clearly limits the system, in this case the memory fetches the entire 16-byte cache line before transmitting it in a single cycle. Both address and data buses are used for only one cycle per transaction. Note that the figure for case (d) allows an additional cycle to reaccess the bus, although this might not be needed and is not accounted for in case (c).



Cases (c) and (d) are interesting, since the bus bandwidth exceeds the memory bandwidth; for instance, in case (d), the memory is busy for seven cycles (four cycles to access the first word and three cycles to assess the remaining words) but the bus is busy for only one cycle. In both of these cases, the “bus”–memory situation is memory limited since that is where the contention will develop.

## 5.4 SOC STANDARD BUSES

Two commonly used SOC bus standards are the Advanced Microcontroller Bus Architecture (AMBA) bus developed by ARM and the CoreConnect bus developed by IBM. The latter has been adopted in Xilinx’s Virtex platform FPGA families.

### 5.4.1 AMBA

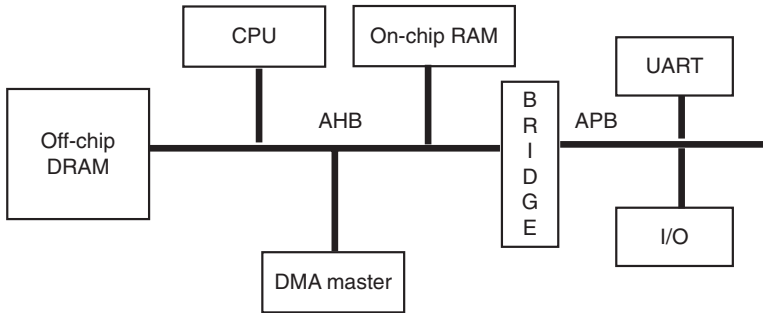
The AMBA, introduced in 1997, had its origin from the ARM processor, which is one of the most successful SOC processors used in the industry. The AMBA bus is based on traditional bus architecture employing two levels of hierarchy. Two buses are defined in the AMBA specification [22]:

- The Advanced High-Performance Bus (AHB) is designed to connect embedded processors, such as an ARM processor core, to high-performance peripherals, direct memory access (DMA) controllers, on-chip memory, and interfaces. It is a high-speed, high-bandwidth bus architecture that uses separate address, read, and write buses. A minimum of 32-bit data operation is recommended in the standard and data widths are extendable to 1024 bits. Concurrent multiple master/slave operations are supported. It also supports burst mode data transfers and split transactions. All transactions on the AHB bus are referenced to a single clock edge, making system-level design easy to understand.
- The Advanced Peripheral Bus (APB) has a lower performance than the AHB bus, but is optimized for minimal power consumption and has reduced interface complexity. It is designed for interfacing to slower peripheral modules.

A third bus, the Advanced System Bus (ASB), is an earlier incarnation of the AHB, designed for lower performance systems using 16/32-bit microcontrollers. It is used where cost, performance, and complexity of the AHB is not justified.

The AMBA bus was designed to address a number of issues exposed by users of the ARM processor bus in SOC integration. The goals achieved by its design are [95]:

1. *Modular Design and Design Reuse.* Since the ARM processor bus interface is extremely flexible, inexperienced designers could inadvertently create inefficient or even unworkable designs by using ad hoc bus and control logic. The AMBA specification encourages a modular design methodology that supports better design partitioning and design reuse.
2. *Well-Defined Interface Protocol, Clocking, and Reset.* AMBA specifies a low-overhead bus interface and clocking structure that is simple yet flexible. The performance of the AMBA bus is enhanced by its multimaster, split transaction, and burst mode operations.
3. *Low-Power Support.* One of the attractions of the ARM processor when compared with other embedded processor cores is its power efficiency. The two-level partitioning of the AMBA buses ensures energy-efficient designs in the peripheral modules, which fits well with the low-power CPU core.



**Figure 5.2** A typical AMBA bus-based system [95].

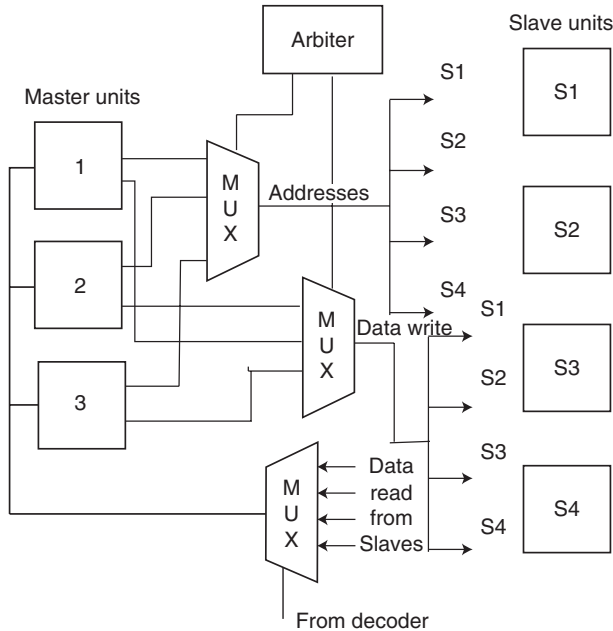
4. *On-Chip Test Access.* AMBA has an optional on-chip test access methodology that reuses the basic bus infrastructure for testing modules that are connected to the bus.

**The AHB** Figure 5.2 depicts a typical system using the AMBA bus architecture. The AHB forms the system backbone bus on which the ARM processor, the high-bandwidth memory interface and random-access memory (RAM), and the DMA devices reside. The interface between the AHB bus and the slower APB bus is through a bus bridge module.

The AMBA AHB bus protocol is designed to implement a multimaster system. Unlike most bus architectures designed for PCB-based systems, the AMBA AHB bus avoids tristate implementation by employing a central multiplexer interconnect scheme. This method of interconnect provides higher performance and lower power than using tristate buffers. All bus masters assert the address and control signals, indicating the type of transfer each master requires. A central arbiter determines which master has its address and control signal routed to all the slaves. A central decoder circuit selects the appropriate read data and response acknowledge signal from the slave that is involved in the transaction. Figure 5.3 depicts such a multiplexer interconnect scheme for a system with three masters and four slaves.

Transactions on the AHB bus involve the following steps:

- *Bus Master Obtains Access to the Bus.* This process begins with the master asserting a request signal to the arbiter. If more than one master simultaneously requests the control of the bus, the arbiter determines which of the requesting masters will be granted the use of the bus.
- *Bus Master Initiates Transfer.* A granted bus master drives the address and control signals with the address, direction, and width of the transfer. It also indicates whether the transaction is part of a burst in the case of burst mode operation. A write data bus operation moves data from the master to a slave, while a read data bus operation moves data from a slave to the master.



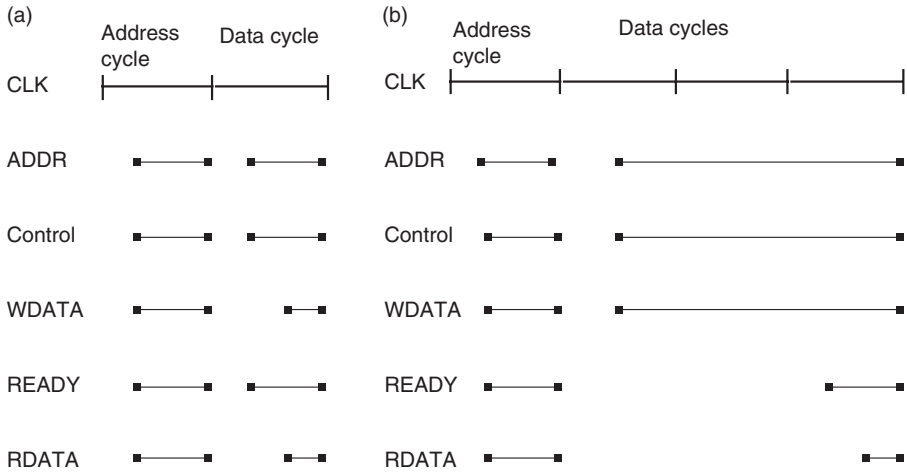
**Figure 5.3** Multiplexor (MUX) interconnection for a three masters/four slaves system [22].

- *Bus Slave Provides a Response.* A slave signals to the master the status of the transfer such as whether it was successful, if it needs to be delayed, or that an error occurred.

Figure 5.4a depicts a basic AHB transfer cycle. An AHB transfer consists of two distinct phases: the address phase and the data phase. The master asserts the address (ADDR) and control signals on the rising edge of the clock (CLK) during the address phase, which always lasts for a single cycle. The slave then samples the address and control signals and responds accordingly during the data phase to a data read (RDATA) or write (WDATA) operation, and indicates its completion with the READY signal. A slave may insert wait states into any transfer by delaying the assertion of READY as shown in Figure 5.4b. For a write operation, the bus master holds the data stable throughout the extended data cycles. For a read transfer the slave does not provide valid data until the last cycle of the data phase.

The AHB bus is a pipelined (tenured) bus. Therefore, the address phase of any transfer can occur during the data phase of a previous transfer. This overlapping pipeline feature allows for high-performance operation.

**The APB** The APB is optimized for minimal power and low complexity instead of performance. It is used to interface to peripherals, which are low bandwidth.



**Figure 5.4** A simple AHB transfer [22]. (a) No wait states in transfer; (b) with wait states during transfer.

The operation of the APB is straightforward and can be described by a state diagram with three states. The APB either stays in the Idle state, or loops around the Setup state and the Enable state during data transfer.

### 5.4.2 CoreConnect

As in the case of AMBA bus, IBM's CoreConnect Bus is an SOC bus standard designed around a specific processor core, the PowerPC, but it is also adaptable to other processors. The CoreConnect Bus and the AMBA bus share many common features. Both have a bus hierarchy to support different levels of bus performance and complexity. Both have advanced bus features such as multiple master, separate read/write ports, pipelining, split transaction, burst mode transfer, and extendable bus width.

The CoreConnect architecture provides three buses for interconnecting cores, library macros, and custom logic:

- processor local bus (PLB),
- on-chip peripheral bus (OPB),
- device control register (DCR) bus.

Figure 5.5 illustrates how the CoreConnect architecture can be used in an SOC system built around a PowerPC. High-performance, high-bandwidth blocks such as the PowerPC 440 CPU core, the PCI-X bus bridge, and the PC133/DDR133 (DDR1 with a 133 MHz bus) synchronous dynamic RAM (SDRAM) Controller are connected together using the PLB, while the OPB hosts lower data rate on-chip peripherals. The daisy-chained DCR bus provides a

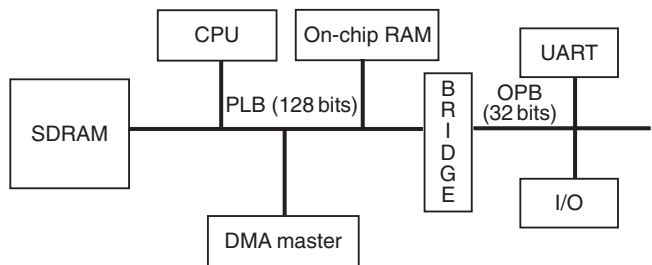


Figure 5.5 A CoreConnect-based SOC [123].

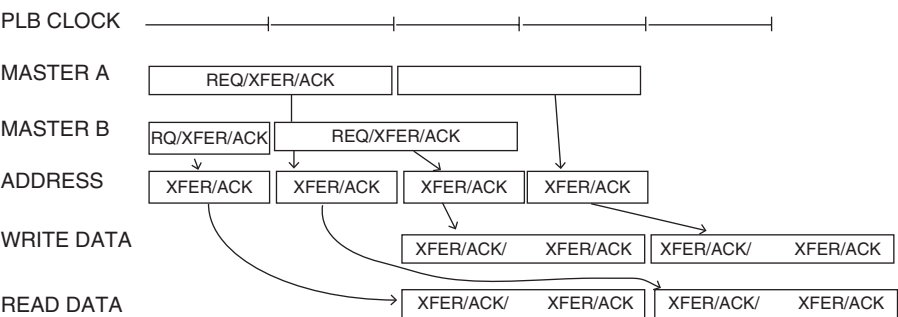


Figure 5.6 PLB transfer protocol [123].

relatively low-speed datapath for passing configuration and status information between the PowerPC 440 CPU core and other on-chip modules.

**The PLB** The PLB is used for high-bandwidth, high-performance, and low-latency interconnections between the processors, memory, and DMA controllers [123]. It is a fully synchronous, split transaction bus with separate address, read, and write data buses, allowing two simultaneous transfers per clock cycle. All masters have their own Address, Read Data, Write Data, and control signals called transfer qualifier signals. Bus slaves also have Address, Read Data, and Write Data buses, but these buses are shared.

PLB transactions, as in the AMBA AHB, consist of multiple phases that may last for one or more clock cycles, and involve the address and data buses separately. Transactions involving the address bus have three phases: request (RQ), transfer (XFER), and address acknowledge (ACK). A PLB transaction begins when a master drives its address and transfer qualifier signals and requests ownership of the bus during the request phase of the address tenure. Once the PLB arbiter grants bus ownership, the master's address and transfer qualifiers are presented to the slave devices during the transfer phase. The address cycle terminates when a slave latches the master's address and transfer qualifiers during the address acknowledge phase.

Figure 5.6 illustrates two deep read and write address pipelining along with concurrent read and write data tenures. Master A and Master B represent the

state of each master's address and transfer qualifiers. The PLB arbitrates between these requests and passes the selected master's request to the PLB slave address bus. The trace labeled Address Phase shows the state of the PLB slave address bus during each PLB clock.

Each data beat in the data tenure has two phases: transfer and acknowledge. During the transfer phase the master drives the write data bus for a write transfer or samples the read data bus for a read transfer. As shown in Figure 5.6, the first (or only) data beat of a write transfer coincides with the address transfer phase.

**Split Transaction** The PLB address, read data, and write data buses are decoupled, allowing for address cycles to be overlapped with read or write data cycles, and for read data cycles to be overlapped with write data cycles. The PLB split bus transaction capability allows the address and data buses to have different masters at the same time. Additionally, a second master may request ownership of the PLB, via address pipelining, in parallel with the data cycle of another master's bus transfer. This situation is illustrated in Figure 5.6, with the dependence of various signals indicated by arrows.

**The OPB** The OPB is a secondary bus designed to alleviate system performance bottlenecks by reducing capacitive loading on the PLB [126]. Peripherals suitable for attachment to the OPB include serial ports, parallel ports, UARTs, GPIO (general purpose I/O), timers, and other low-bandwidth devices. The OPB is more sophisticated than the AMBA APB. It supports multiple masters and slaves by implementing the address and data buses as a distributed multiplexer. This type of structure is suitable for the less data-intensive OPB bus and allows peripherals to be added to a custom core logic design without changing the I/O on either the OPB arbiter or existing peripherals. Figure 5.7 shows one method of structuring the OPB address and data buses. Both masters and slaves provide enable control signals for their outbound buses. By requiring that each unit provide this signal, the associated bus combining logic can be strategically placed throughout the chip. As shown in the figure, either of the masters is capable of providing an address to the slaves, whereas both masters and slaves are capable of driving and receiving the distributed data bus.

Table 5.3 shows a comparison between the AMBA and CoreConnect bus standards.

### 5.4.3 Bus Interface Units: Bus Sockets and Bus Wrappers

Using a standard SOC bus for the integration of different reusable IP blocks has one major drawback. Since standard buses specify protocols over wired connections, an IP block that complies with one bus standard cannot be reused with another block using a different bus standard. One approach to alleviate this is to employ a hardware “socket,” which is an example of a bus wrapper in Section 5.2, to separate the interconnect logic from the IP

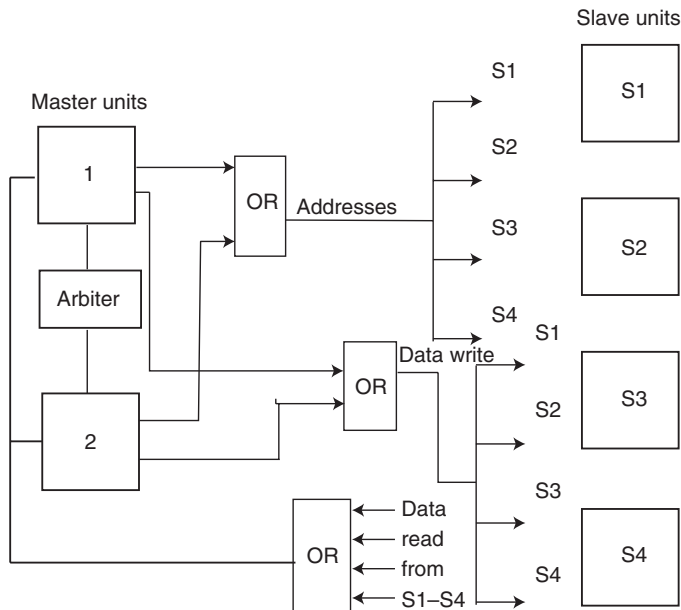


Figure 5.7 The on-chip peripheral bus (OPB) [126].

TABLE 5.3 Comparison between CoreConnect and AMBA Architectures [198]

|                   | IBM CoreConnect PLB                                                                                                                            | ARM AMBA 2.0 AMBA High-Performance Bus                                                                    |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Bus architecture  | 32, 64, and 128 bits, extendable to 256 bits                                                                                                   | 32, 64, and 128 bits                                                                                      |
| Data buses        | Separate read and write                                                                                                                        | Separate read and write                                                                                   |
| Key capabilities  | Multiple bus masters<br>Four-deep read pipelining, two-deep write pipelining<br>Split transactions<br>Burst transfers<br>Line transfers<br>OPB | Multiple bus masters<br>Pipelining<br>Split transactions<br>Burst transfers<br>Line transfers<br>AMBA APB |
| Masters supported | Supports multiple masters                                                                                                                      | Single master: The APB bridge                                                                             |
| Bridge function   | Master on PLB or OPB                                                                                                                           | APB master only                                                                                           |
| Data buses        | Separate read and write                                                                                                                        | Separate or three-state                                                                                   |

core using a well-defined IP core protocol that is independent of the physical bus protocol. Core-to-core communication is therefore handled by the interface wrapper. This approach is taken by the Virtual Socket Interface Alliance (VSIA) [44] with their virtual component interface (VCI) [249], and by Sonics Inc. employing the Open Core Protocol (OCP) and Silicon Backplane  $\mu$ Network [225].

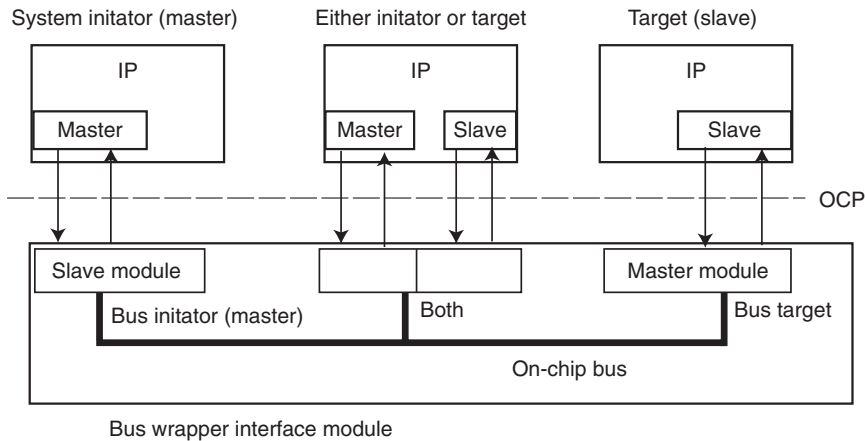


VSIA proposes a set of standards and interfaces known as virtual socket interface (VSI) that enables system-level interaction on a chip using pre-designed blocks (called virtual components [VCs]) [249]. This encourages designs using a component paradigm. The VCs, which are effectively IP blocks that conform to the VSI specifications, can be one of three varieties. *Hard* VCs consist of placed and routed gates with all silicon layers defined. It has predictable performance, area usage, and power consumption, but offers no flexibility. *Soft* VCs are designed in some hardware description language representation, which are mapped to physical design through synthesis, placement, and routing. They can be easily modified but generally take more effort to integrate and verify in the SOC design as well as having less predictable performance. Finally, *firm* VCs offer a compromise between the two. They come in the form of generators or partially placed library blocks that require final routing and/or placement adjustment. This form of VCs provides more predictable performance than soft VCs, but still offers some degree of flexibility in aspect ratio and configuration.

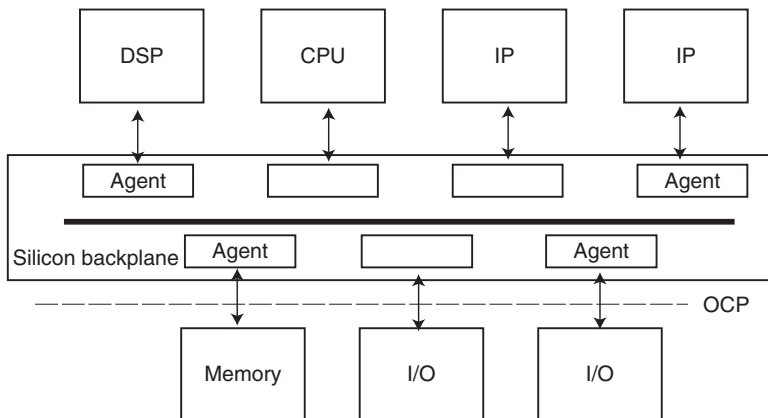
In order to connect these different VCs together, VSIA has developed a VCI specification to which other proprietary buses can interface. By following the VCI specification, a designer can take a VC and integrate it with any of several buses in order to meet system performance requirements. The VCI standard specifies a family of protocols. Currently three protocols are defined: the peripheral VCI (PVCI), the basic VCI (BVCI), and the advanced VCI (AVCI) [249]. The PVCI is a low-performance protocol where the request and the response data transfer occur during a single control handshake transaction. It is therefore not a split-transaction protocol. The BVCI employs a split-transaction protocol, but responses must arrive in order. In other words, the response data must be supplied in the same order in which the initiator generated the requests. The AVCI is similar to the BVCI, but out-of-order transactions are allowed. Requests are tagged and transactions can be interleaved and reordered.

In addition to the specification of the VCI, VSIA also specifies a number of abstraction layers to define the representation views required to integrate a VC into an SOC design [44]. The idea is that if both the IP block provider (VC provider) and the system integrator (VC integrator) conform to the VSI specifications at all levels of abstraction, SOC designs using an IP component paradigm can proceed with lower risk of errors.

An alternative to VCI is the OCP promoted by the Open Core Protocol International Partnership (OCP-IP) [188]. The OCP defines a point-to-point interface between two communicating entities such as two IP cores using a core-centric protocol. An interface implementing the OCP assumes the attributes of a *socket*, which, as explained earlier, is effectively a bus wrapper that allows interfacing to the target bus. A system consisting of three IP core modules using the OCP and bus wrappers is shown in Figure 5.8. One module is a system initiator, one is a system target, and another is both initiator and target.



**Figure 5.8** A three-core system using OCP and bus wrappers [225].

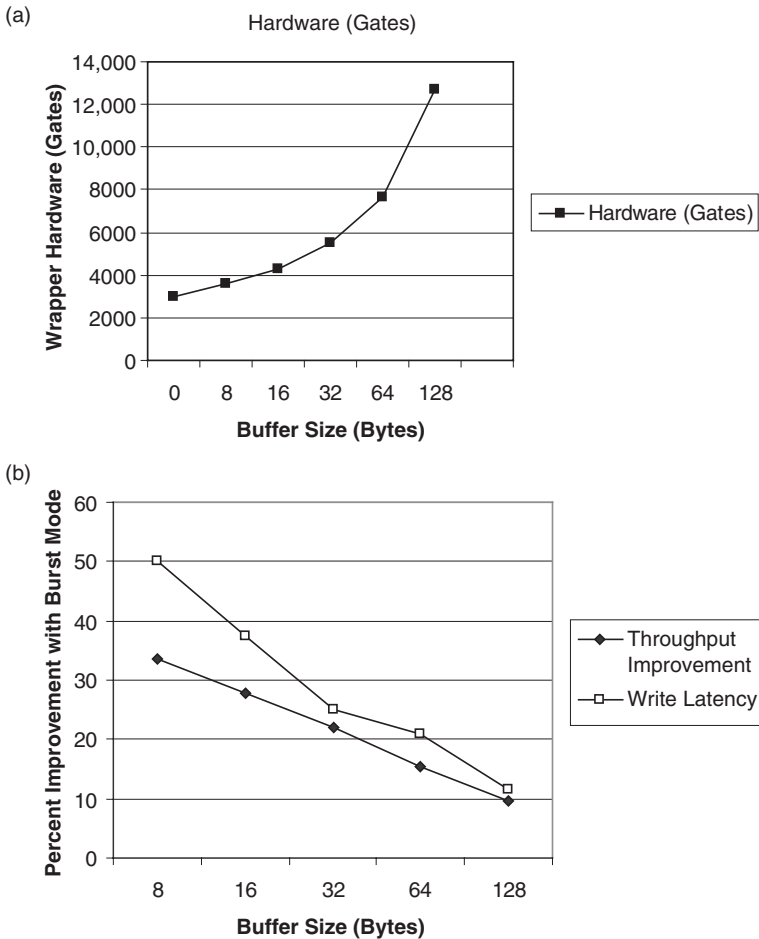


**Figure 5.9** Sonics  $\mu$ Network configuration [225]. DSP, digital signal processor.

Another layer of interconnection can be made above the OCP in order to help IP integration further. Sonics Inc. proposes their proprietary SiliconBackplane Protocol that seamlessly glues together IP blocks that uses the OCP. The communication between different blocks takes place over the Silicon Backplane  $\mu$ Network, which has a scalable bandwidth of 50–4000 MB/s. Figure 5.9 depicts how the Sonics  $\mu$ Network components are connected together [225].

Bus interface units using the wrapper-based approach have been demonstrated to reduce the design time of SOC, but at a cost in terms of gates and latency. Attaching simple wrapper hardware increases the access latencies and incurs a hardware overhead of 3–5 K gates [160].

In addition, bus interface units can include first-in–first-out (FIFO) buffers to improve performance. Figure 5.10 shows the amount of hardware overhead



**Figure 5.10** (a) Hardware overhead of write buffers; (b) performance impact of buffer for burst mode transfer [9].

incurred and performance improvement achieved by employing write data buffers in a bus interface unit [9].

The write buffer provides several cycles improvement in latency and, depending on the data size, more than 10% improvement in throughput.

## 5.5 ANALYTIC BUS MODELS

### 5.5.1 Contention and Shared Bus

Contention occurs wherever two or more units request a shared resource that cannot supply both at the same time. When contention occurs, either (1) it

delays its request and is idle until the resource is available or (2) it queues its request in a buffer and proceeds until the resource is available. Case (2) is only possible when the requested item is not logically essential to program execution (as in a cache prefetch, for example).

Whether we need to analyze the bus as a source of contention depends on its maximum (or offered) bandwidth relative to the memory bandwidth. As contention and queues develop at the “bottleneck” in the system, the most limiting resource is the source of the contention, and other parts of the system simply act as delay elements. Thus buses must be analyzed for contention when they are more restrictive (have less available bandwidth) than memory.

Buses often have no buffering (queues), and access delays cause immediate system slowdown. The analysis on the effects of bus congestion depends on the access type and buffering.

Generally there are two types of access patterns:

1. *Requests without Immediate Resubmissions.* The denied request returns with the same arrival distribution as the original request. Once a request is denied, processing continues despite the delay in the resubmission of the request. This is the case of a cache line prefetch, which is not currently required for continued program execution.
2. *Requests Are Immediately Resubmitted.* This is a more typical case, when multiple independent processors access a common bus. A program cannot proceed after a denied request. It is immediately resubmitted. The processor is idle until the request is honored and serviced.

### 5.5.2 Simple Bus Model: Without Resubmission

In the following, we assume that each request occupies the bus for the same service time (e.g.,  $T_{\text{line access}}$ ). Even if we have two different types of bus users (e.g., word requests and line requests on a single line or [dirty] double line requests), most cases are reasonably approximated by simple computation of the per-processor average (offered) bus occupancy,  $\rho$ , given by:

$$\rho = \frac{\text{bus transaction time}}{\text{processor time} + \text{bus transaction time}}.$$

The processor time is the mean time the processor needs to compute before making a bus request. Of course, it is possible for the processor to overlap some of its compute time with the bus time. In this case, the processor time is the net nonoverlapped time between bus requests. In any event,  $\rho \leq 1$ .

The simplest model for  $n$  processors accessing a bus is given by:

$$\text{Prob (processor does not access bus)} = 1 - \rho$$

$$\begin{aligned}\text{Prob (bus is busy)} &= (1 - \rho)^n \\ &= \text{fraction of bus bandwidth realized} = B(\rho, n).\end{aligned}$$

The fraction of bandwidth realized times the maximum bus bandwidth gives the realized (or achieved) bus bandwidth,  $Bw$ .

The achieved bandwidth fraction (achieved occupancy) per processor ( $\rho_a$ ) is given by:

$$\begin{aligned}n\rho_a &= B(\rho, n) \\ \rho_a &= \frac{B(\rho, n)}{n}\end{aligned}$$

A processor slows down by  $\rho_a/\rho$  due to bus congestion.

### 5.5.3 Bus Model with Request Resubmission

A model that supports request resubmission involves a more complex analysis and requires an iterative solution. There are several solutions, each providing similar results. The solution provided by Hwang and Briggs [122] is an iterative pair of equations:

$$a = \frac{\rho}{\rho + (\rho_a/\rho)(1 - \rho)}$$

and

$$n\rho_a = 1 - (1 - a)^n,$$

where  $a$  is the actual offered request rate. To find a final  $\rho_a$ , initially set  $a = \rho$  to begin the iteration. Convergence usually occurs within four iterations.

### 5.5.4 Using the Bus Model: Computing the Offered Occupancy

The model in the preceding section does not distinguish among types of transactions. It just requires the mean bus transaction time, which is the average number of cycles that the bus is busy managing a transaction. Then the issue is finding the offered occupancy,  $\rho$ .

The offered occupancy is the fraction of the time that the bus would be busy if there were no contention among transactions (bounded by 0.0 and 1.0). In order to find this, we need to determine the mean time for a bus transaction and the compute time between transactions.

The nature of the processor initiating the transaction is another factor. Simple processors make *blocking* transactions. In this case the processor is idle after the bus request is made and resumes computation only after the bus transaction is complete. The alternative for more complex processors is a

*buffered* (or nonblocking) transaction. In this case the processor continues processing after making a request, and may indeed make several requests before completion of an initial request. Depending on the system configuration, there are two common cases:

1. *A Single Bus Master with Blocking Transactions.* In this case there is no bus contention as the processor waits for the transaction to complete. Here the achieved occupancy,  $\rho_a$ , is the same as the offered occupancy, and  $\rho = \rho_a = (\text{bus transaction time})/(\text{compute time} + \text{bus transaction time})$ .
2. *Multiple (n) Bus Masters with Blocking Transactions.* In this case the offered occupancy is simply  $n\rho$  where  $\rho$  is as in case (1). Now contention can develop so we use our bus model to determine the achieved occupancy,  $\rho_a$ .

**Example.** Suppose a processor has bus transactions that consist of cache line transfers. Assume that 80% of the transactions move a single line and occupy the bus for 20 cycles and 20% of the transactions move a double line (as in dirty line replacement), which takes 36 cycles. The mean bus transaction time is 23.2 cycles. Now assume that a cache miss (transaction) occurs every 200 cycles.

In case (1), the bus is occupied:  $\rho = \rho_a = 23.2/223.2 = 0.10$ ; there is no contention, but the bus causes a system slow down, as discussed below.

In case (2), suppose we have four processors. Now the offered occupancy is  $\rho = 0.104$  and we use our model to find the contention time. Initially we set  $a = \rho = 0.104$ ,  $n\rho_a = 1 - (1 - a)^n = 1 - (1 - 0.104)^4$ ; now we find  $\rho_a$  and substitute the value of  $\rho_a$  for  $a$  and continue.

So initially,  $\rho_a = 0.089$ ; after the next iteration,  $\rho_a = 0.010$ ; and after several iterations,  $\rho_a = 0.095$ . We always achieve less than what is offered and the difference is delay due to contention. So:

$$\rho_a = 0.095 = \frac{\text{bus transaction time}}{\text{compute time} + \text{bus transaction time} + \text{contention time}}.$$

Solving for the contention time, we get about 21 cycles.

### 5.5.5 Effect of Bus Transactions and Contention Time

There are two separate effects of bus delays on overall system performance. The first is the obvious case of blocking, which simply inserts a transaction delay into the program execution. The second effect is due to contention. Contention reduces the rate of transaction flow into the bus and memory. This reduces performance proportionally.

In the case of blocking the processor simply slows down by the amount of the bus transaction. So the relative performance compared to an ideal processor with no bus transactions is:

$$\text{Relative performance} = \frac{\text{compute time}}{\text{compute time} + \text{bus transaction time}}.$$

In the case (1) example the processor slows down by  $200/223.3 = 0.896$ .

Contention, when present, adds additional delay. In case (2) the individual processor slows down by  $200/(223.2 + 21) = 0.819$ . The result of contention is that it simply slows down the system (without contention) by the ratio of  $\rho_a/\rho$ . The supply of transactions is reduced by this ratio.

# 6 Customization and Configurability

## 6.1 INTRODUCTION

To broaden SOC applicability while reducing cost, one can adopt a common hardware platform that can be customized to improve efficiency for specific applications. This chapter looks at different *customization* technologies, particularly those based on *configurability*. Here configurability covers both *one-time configurability*, when application-oriented customization takes place once either before or after chip fabrication, and *reconfigurability*, when customization takes place multiple times after chip fabrication.

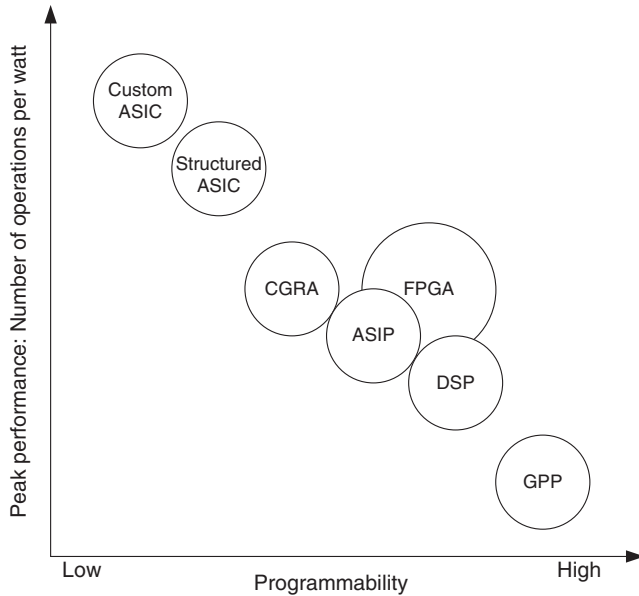
Customization opportunities at design time, particularly those exploited in device fabrication, often result in high performance but at the expense of flexibility when the design is deployed. Such postfabrication flexibility is achieved by devices with various degrees of programmability, including coarse-grained reconfigurable architectures (CGRAs), application-specific instruction processors (ASIPs), fine-grained field-programmable gate arrays (FPGAs), digital signal processors (DSPs), and general-purpose processors (GPPs). The trade-off between programmability and performance is shown in Figure 6.1, which is introduced in Chapter 1.

Structured ASIC (application-specific integrated circuit) technology supports limited customization before fabrication compared with custom ASIC technology. In Figure 6.1, the ASIPs are assumed to be customized at fabrication in ASIC technology. ASIPs can also be customized at compile time if implemented in FPGA technology as a soft processor; a customizable ASIP processor will be presented in Section 6.8.

There are many ways of customizing SOC designs, and this chapter focuses on three of them:

1. customization of instruction processors (Sections 6.4 and 6.8), illustrating how (a) availability of processor families and (b) generation of application-specific processors can offer architectural optimizations such as very long instruction word (VLIW), vectorization, fused operation,





**Figure 6.1** A simplified comparison of different technologies: programmability versus performance. GPP stands for general-purpose processor, while CGRA stands for coarse-grained reconfigurable architecture.

and multithreading to meet requirements in performance, area, energy efficiency, and costs;

2. customization of reconfigurable fabrics (Sections 6.5 and 6.6), showing that fine-grained reconfigurable functional units (FUs) and the related interconnect resources are versatile but incur large overheads—hence coarse-grained blocks are increasingly adopted to reduce such overheads;
3. customization techniques for optimizing implementations, such as instance-specific design (Section 6.7) and run-time reconfiguration strategies (Section 6.9), together with methods for assessing related trade-offs in performance, size, power, and energy efficiency.

Other customization methods, such as those based on multiprocessors, would not be treated in detail. Pointers to references on various related topics are included in Section 6.10.

## 6.2 ESTIMATING EFFECTIVENESS OF CUSTOMIZATION

It is important to be able to estimate and compare the effectiveness of different ways of customization applied to part of a design. The method is simple.

For a given metric such as delay, area, or power consumption, assume that  $\alpha$  is the factor of improvement applicable to a fraction  $\beta$  of a design. So the metric is improved by:

$$(1 - \beta) + \alpha \times \beta.$$

This metric is reminiscent of the parallel processor analysis of G. Amdahl. As an example, consider the well-known 90:10 rule: a good candidate for acceleration is the case when 10% of the code takes 90% of the time. What happens if that 10% of the code can be accelerated  $k$  times?

From the expression above with  $\alpha = 1/k$ ,  $\beta = 0.9$ , we obtain  $(k + 9)/10k$ . Assuming  $k = 10$ , the execution time is reduced to 19% of the original, resulting in a speedup of 5.26 times.

However, if the effect of the code that can be accelerated is reduced from 90% of the time to only 60% of the time, we can find that the speedup is only 2.17 times—almost halved.

Note that the amount of code, which provides an estimate of the amount of effort required for customizing it, does not affect the above result; it is the effect of the customizable fraction on the metric that matters, not the fraction itself.

This method can be applied in various ways. As another example, consider the use of embedded coarse-grained blocks to customize a fine-grained reconfigurable fabric, as we shall explain in a later section. Assume that 50% of the fine-grained fabric can be replaced by coarse-grained blocks, which are three times more efficient in speed and 35 times more efficient in area. One can find that the design could be improved up to 50% faster with its area reduced by half.

There are, however, several reasons against customizability. Tools for customizable processors such as performance profilers and optimizers are often not as mature as those for noncustomizable processors; backwards compatibility and verification can also become an issue. One approach is to develop customizable processors that are compatible with existing noncustomizable building blocks, such that there is interoperability between customizable and noncustomizable technologies [94].

### 6.3 SOC CUSTOMIZATION: AN OVERVIEW

Customization is the process of optimizing a design to meet application requirements and implementation constraints. It can take place at design time and at run time. Design time has two components: fabrication time and compile time. During fabrication time, a physical device is constructed. If this device is configurable, then after fabrication it can be customized by a program produced at compile time and executed at run time.

There are three common means of implementing computations: standard instruction processors, ASICs, and reconfigurable devices.

1. For standard instruction processors such as those from ARM, AMD, and Intel, fabrication-time customization produces a device supporting a fixed instruction set architecture, and compile-time customization produces instructions for that architecture; run-time customization corresponds to locating or generating appropriate code for execution at run time.
2. For ASICs, much of the customization to perform application functions takes place at fabrication time. Hence the high efficiency but low flexibility, since new functions not planned before fabrication cannot easily be added. Structured ASICs, such as gate array or standard-cell technologies, reduce design effort by limiting the options of customization to the user to, for instance, the metal layers. One-time customization of antifuse technology can be performed in the field.
3. Reconfigurable devices generally include FPGA and complex programmable logic device (CPLD) technology, as well as instruction processors coupled with a reconfigurable fabric to support custom instructions [25]. In this case, fabrication-time customization produces a device with a reconfigurable fabric, typically containing reconfigurable elements joined together by reconfigurable interconnections. At compile time, configuration information is produced from a design description for customizing the device at appropriate instants at run time.

The standard instruction processors are general purpose. There are, however, opportunities to customize the instruction set and the architecture for a specific application. For instance, the standard instruction set can be customized to remove unused instructions or to include new instructions that would result in improved performance. Custom instruction processors can be customized during fabrication in ASIC technology or during configuration in reconfigurable hardware technology.

- Processors that are customized during fabrication include those from ARC and Tensilica. Typically some of the building blocks are hardwired at fabrication to support, for instance, domain-specific optimizations, including instructions that are customized for specific applications. To reduce risk, there are often reconfigurable prototypes before designs are implemented in ASIC technology.
- For soft processors such as MicroBlaze [259] from Xilinx or Nios [11] from Altera, the challenge is to support instruction processors efficiently using resources in a reconfigurable fabric such as an FPGA. Efficiency can be improved by exploiting device-specific features or run-time reconfigurability [213].
- Another alternative is to implement the instruction processor in ASIC technology, and a suitable interface is developed to enable the processor to benefit from custom instructions implemented in a reconfigurable fabric. An example is the software configurable processor from Stretch [25], which consists of the Xtensa instruction processor from Tensilica

and a coarse-grained reconfigurable fabric. These devices can offer higher efficiency than FPGAs when an application requires resources that match their architecture.

A key consideration for deciding which technology to use is volume. Recall from Chapter 1 that the product cost usually has a fixed or nonrecurring component that is independent of volume and has a variable or recurrent component that varies with volume. Reconfigurable technologies like FPGAs have little fixed cost, but have a higher unit cost than ASIC technologies. Hence, below a certain volume threshold, reconfigurable technologies offer the lowest cost—and this threshold moves in favor of reconfigurable technologies with time, since the cost of mask sets and other fixed fabrication costs increase rapidly with each new generation of technology.

There are various ways of classifying a customizable SOC. A customizable SOC typically consists of one or more processors, reconfigurable fabrics, and memories. One way is to classify such SOC's according to the coupling between the reconfigurable fabric and the processor [61].

Figure 6.2a shows the reconfigurable fabric attached to the system bus. Figure 6.2b illustrates the situation when the reconfigurable fabric is a coprocessor of the CPU, with a closer coupling between them than the ones in Figure 6.2a.

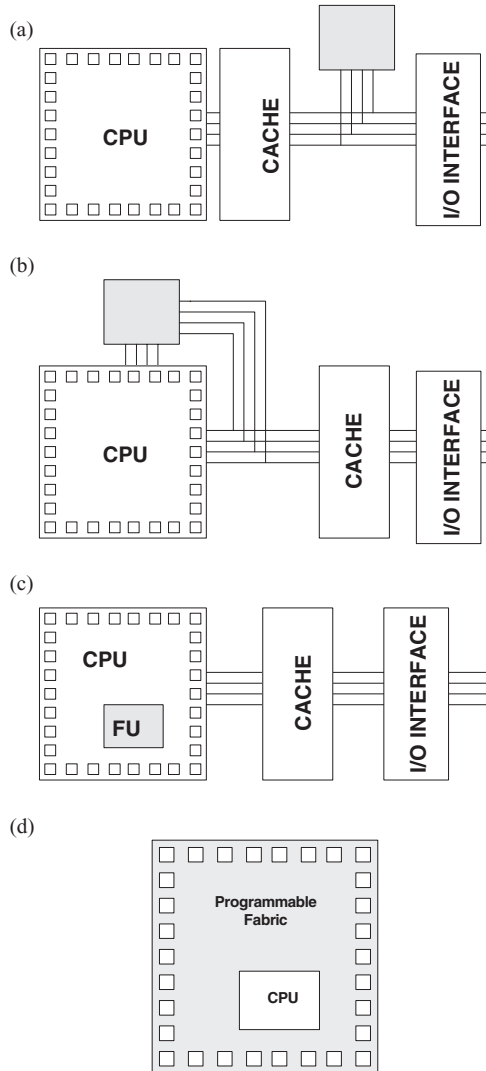
Next, Figure 6.2c shows an architecture in which the processor and the fabric are tightly coupled. In this case, the reconfigurable fabric is part of the processor itself, perhaps forming a reconfigurable subunit that supports custom instructions. An example of this organization is the software configurable processor from Stretch.

Figure 6.2d shows another organization. In this case, the processor is embedded in the programmable fabric. The processor can either be a “hard” core [261] or can be a “soft” core, which is implemented using the resources of the reconfigurable fabric itself; examples include the MicroBlaze and the Nios processors mentioned earlier.

It is also possible to integrate configurable analog and digital functionality on the same chip. For instance, the PSoC (programmable system on a chip) device [68] from Cypress has an array of analog blocks that can be configured as various combinations of comparators, filters, and analog-to-digital converters, with programmable interconnects. The inputs and outputs of these and the reconfigurable digital blocks can be flexibly routed to the input/output (I/O) pins. Moreover, these blocks can be reconfigured to perform different functions when the system is operating.

## 6.4 CUSTOMIZING INSTRUCTION PROCESSORS

Microprocessors in desktop machines are designed for general-purpose computation. Instruction processors in an SOC are often specialized for particular



**Figure 6.2** Four classes of customizable SOC [61; 244]. The shaded box denotes the reconfigurable fabric. (a) Attached processing unit; (b) coprocessor; (c) reconfigurable FU; (d) processor embedded in a reconfigurable fabric.

types of computations such as media processing or data encryption. Hence they can benefit from customization that is highly specific to their intended function, eliminating hardware elements that would not be needed. Such customization usually takes place before fabrication, although many techniques can also be applied to the design of soft processors. Customization allows designers to optimize their designs to meet requirements such as those

in speed, area, power consumption, and accuracy, while improving product differentiation.

#### 6.4.1 Processor Customization Approaches

There are two main approaches for providing customized processors. The first approach is to provide families of processors customized for specific application domains. For example, ARM provides the Cortex-A Series for supporting applications with demanding computation, the Cortex-R Series for real-time processing, the Cortex-M series for microcontrollers in embedded applications (with the Cortex-M1 processors optimized for FPGA implementation), and the SecurCore processors for tamper-resistant smart cards. Each series contains a range of processors with different characteristics, allowing designers to choose one that best fits their requirements in function and performance, including power and energy consumption.

The second approach is to provide the capability of generating customized processors. Companies such as ARC and Tensilica provide design tools to allow SOC designers to configure and extend a processor, either through a graphical user interface or through tools based on an architecture description language. Such tools enable designers to add only the features they need while deleting the features they do not need. In addition, they allow designers to extend the architecture of the core by adding custom instructions, allowing further optimization of the processor for the end application.

To help optimize the design for size, power, and application performance, some SOC tools provide guidelines for final silicon area and memory requirements. Designers are able to configure features around the core, such as the type and size of caches, interrupts, DSP subsystem, timers, and debug components, as well as features within the core, such as the type and size of core registers, address widths, and instruction set options. The aim is to support rapid performance and die-size trade-offs to provide an optimized solution. Specific functions often included in SOC tools are:

- Integration of intellectual properties from various sources.
- Generation of simulation scripts and test benches for system verification.
- Enhancement of software development tools to support, for instance, custom instructions for the customizable processor.
- Automated generation of FPGA designs for an emulation platform.
- Documentation of selected configuration for inclusion in licensees' chip specifications and customer-level documentation.

Such tools can usually configure and deliver a custom processor within minutes. Also, by generating all configuration-specific information required for testing,

downstream development tools, and documentation, the configurator reduces time to silicon and reduces project risk.

Further information about tools and applications of customizable embedded processors can be found in various publications [127, 207].

### 6.4.2 Architecture Description

As processors become increasingly complex, it is useful to provide a significant degree of automation in implementing both the processor and the software tools associated with that processor. Without automation, much design and verification effort is needed to build a processor and its tools from scratch for every application.

To automate development of processors and tools, architecture description languages or processor description languages can help [179]. Such languages should be sufficiently concise and high level for designers to adopt them for different applications; they should also be comprehensive enough to produce realistic, efficient implementations. Many languages have been proposed for describing instruction processors. The goal of these languages is to capture the design of a processor so that supporting tools and the processor itself can be automatically generated.

Architecture description languages can be classified by the description style and by the level and type of automation provided. For example, one can classify architecture description languages as either behavioral, structural, or a combination of the two.

*Behavioral descriptions* are instruction-set centric: The designer specifies the instruction set and then uses tools to generate a compiler, assembler, linker, and simulator. The nML [87] and TIE [240] languages fall into the behavioral category. Automating generation of a compiler backend from a behavioral description is facilitated when the instructions can be expressed as a tree grammar for a code-generator-generator tool such as BURG [100]. Many behavioral languages support synthesis of processor hardware, and synthesis tools are available for the above examples. In the case of TIE, synthesis is simplified since the base processor is fixed and only extensions to this base processor can be specified by designers. The nML “Go” tool designs the processor architecture automatically [86] from the instruction set, inferring structure from explicitly shared resources such as the register file.

The main advantage of a behavioral description is the high level of abstraction: only an instruction set specification is required to generate a custom processor. The main disadvantage is the lack of flexibility in the hardware implementation. Synthesis tools must fix some aspects of the microarchitecture [87] or even the entire base processor [240]. Efficient synthesis from an instruction set alone is a difficult design automation problem when resource sharing [50] is taken into account.

*Structural descriptions* capture the FUs, storage resources, and interconnections of a processor. SPREE [269] is a library built onto C++ that generates

**TABLE 6.1 Features of Some Architecture Description Languages**

| Feature                    | Expression<br>[114] | CUSTARD<br>[77] | LISA<br>[119] | SPREE<br>[269] | nML<br>[239] | TIE<br>[240] |
|----------------------------|---------------------|-----------------|---------------|----------------|--------------|--------------|
| Whole processor            | √                   | √               | √             | √              | √            |              |
| Toolchain<br>configuration | √                   | √               | √             | √              | √            | √            |
| Hardware<br>generation     |                     | √               | √             | √              | √            | √            |
| Memory system              | √                   | √               | √             |                |              |              |
| Behavioral                 | √                   | √               | √             |                | √            | √            |
| Structural                 | √                   |                 | √             | √              |              |              |

FPGA soft processors using a structural description. In particular, the designer can remove instructions or change the implementation of FUs, since SPREE provides a method for connecting functional blocks, with built-in support for common functions such as forwarding networks and interlocks.

The main advantage of structural descriptions is that they can be directly converted into a form suitable for synthesis to hardware. Additionally, most structural description styles maintain the generality of a hardware description language (HDL). This generality provides much scope for describing diverse microarchitectures, for example, superscalar or multithreaded ones. The prime disadvantage is the lower level of abstraction: the designer needs to manually specify FUs and control structures.

We present a summary of existing processor description languages in Table 6.1. For each language, we indicate the description style, the scope (whole processor or just instruction set), and the tools available to automate generation of a processor system.

Some systems, such as LISA [119], cover both structural and behavioral information. This combines the advantages of pure behavioral and structural descriptions, but there is a need to ensure that related behavioral and structural elements are consistent.

All languages except TIE are whole processor descriptions, meaning that the entire processor design is specified, as opposed to just the instruction set. However, many processor description languages are specific to a particular basic processor architecture, such as in-order execution in the case of LISA and nML. Customizable threaded architecture (CUSTARD) [77], which would be covered in Section 6.8, is based on the MIPS instruction set while supporting various customization options, such as the type of multithreading and the use of custom instructions.

Possible tools for such languages include [179]:

- *Model Generation.* Tools for producing hardware prototypes and validation models for checking that the architecture specification captures the requirements.



- *Test Generation.* Tools for producing test programs, assertions, and test benches.
- *Toolkit Generation.* Tools for profiling, exploring, compiling, simulating, assembling, and debugging designs.

### 6.4.3 Identifying Custom Instructions Automatically

Various approaches have been proposed for automatic identification of instruction set extensions from high-level application descriptions. One can cluster related dataflow graph (DFG) nodes heuristically as sequential or parallel templates. Input and output constraints are imposed on the subgraphs to reduce the exponential search space.

Various architectural optimizations—some described in the earlier chapters—can benefit the automatically generated designs, such as [110]:

- VLIW techniques enable a single instruction to support multiple independent operations. A VLIW format partitions an instruction into a number of slots, each of which may contain one of a set of operations. If the instruction set is designed to use VLIW, a source language compiler can use software-pipelining and instruction-scheduling techniques to pack multiple operations into a single VLIW instruction.
- Vector operations increase throughput by creating operations that operate on more than one data element. A vector operation is characterized by the operation it performs on each data element and by the number of data elements that it operates on in parallel, that is, the vector length. For example, a four-wide vector integer addition operation sums two input vectors, each containing four integers, and produces a single result vector of four integers.
- Fused operations involve creating operations composed of several simple operations. A fused operation potentially has one or more of the input operands fixed to a constant value. Using the fused operation in place of the simple operations reduces code size and issue bandwidth, and may reduce register file port requirements. Also, the latency of the fused operation may be lower than the combined latency of the simple operations.

Application of constraint propagation techniques results in an efficient enumerative algorithm. However, the applicability of this approach is limited to DFGs with around 100 nodes. Search space can be further reduced by imposing additional constraints such as single output, or connectivity constraints on the subgraphs.

The identification of instruction set extensions under input and output constraints can be formulated as an integer linear programming problem. Biswas et al. [45] propose an extension to the Kernighan–Lin heuristic based on input and output constraints. Optimality is often limited by either an approximate search algorithm or some artificial constraints—such as I/O constraints—to make subgraph enumeration tractable.

An integer linear programming formulation can replace I/O constraints with the actual data bandwidth constraints and data transfer costs. The instruction set extensions that are generated may have an unlimited number of inputs and outputs. A baseline machine with architecturally visible state registers makes this approach feasible. Promising results are obtained by integrating the data bandwidth information directly into the optimization process, by explicitly accounting for the cost of the data transfers between the core register file and custom state registers [28].

There are many approaches in customizing instruction processors. A technology-aware approach could involve a clustering strategy to estimate the resource utilization of lookup table (LUT)-based FPGAs for specific custom instructions, without going through the entire synthesis process [148]. An application-aware approach could, in the case of video applications, exploit appropriate intermediate representations and loop parallelism [165]. A transformation-aware approach could adopt a method based on combined but phased searching of the source-level transformation design space and the instruction set extension design space [182].

## 6.5 RECONFIGURABLE TECHNOLOGIES

Among various technologies, FPGAs are well known. Their capacity and capability have improved rapidly in the last few years to support high-performance designs. Their low cost and support for rapid development make them ideal for designs requiring fast time to market, as well as for education and student projects.

The following covers the reconfigurable fabric that underpins FPGAs and other reconfigurable devices. The reconfigurable fabric consists of a set of reconfigurable FUs, a reconfigurable interconnect, and a flexible interface to connect the fabric to the rest of the system. We shall review each of these components and show how they have been used in both commercial and academic reconfigurable systems. The treatment here follows that of Todman et al. [244].

In each component of the fabric, there is a trade-off between flexibility and efficiency. A highly flexible fabric is typically larger and slower than a less flexible fabric. On the other hand, a more flexible fabric can better adapt to application requirements. This kind of trade-off can influence the design of reconfigurable systems. A summary of the main features of various architectures can be found in Table 6.2. There are many related devices, such as those from Elixent [82], that we are unable to include due to limited space.

### 6.5.1 Reconfigurable Functional Units (FUs)

Reconfigurable FUs can be classified as either coarse grained or fine grained. A fine-grained FU can typically implement a single function on a single bit,

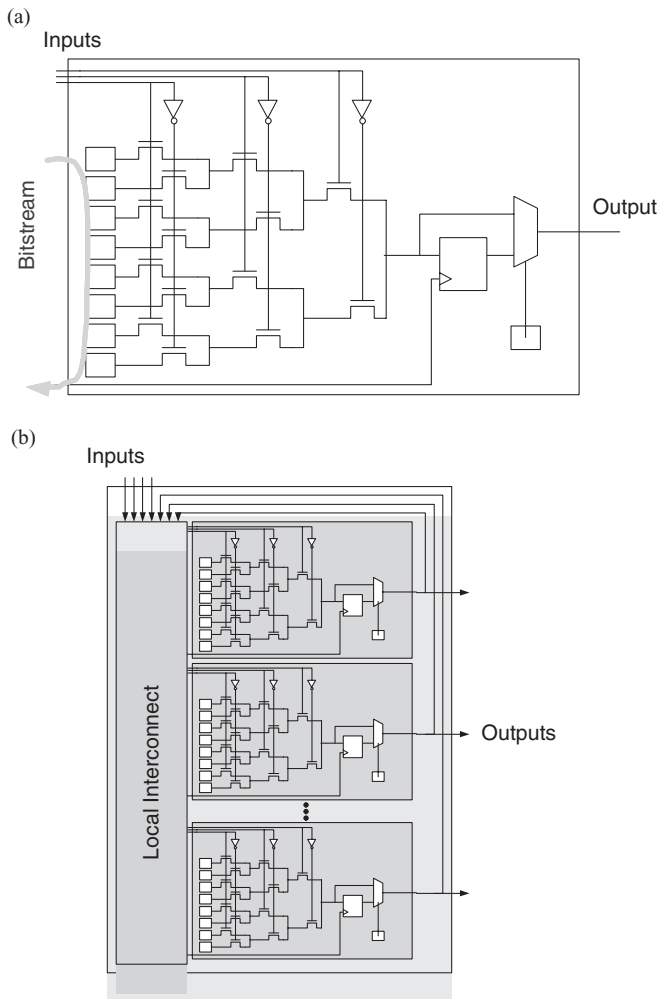
**TABLE 6.2 Comparison of Reconfigurable Fabrics and Devices**

| Fabric or Device                           | Granularity | Base Logic Component                          | Routing Architecture           | Embedded Memory                       | Special Features                                                        |
|--------------------------------------------|-------------|-----------------------------------------------|--------------------------------|---------------------------------------|-------------------------------------------------------------------------|
| Actel Axcelerator [2]                      | Fine        | Four-input mux and inverter                   | Horizontal and vertical tracks | 4-Kbit blocks                         | Antifuse, low-power mode                                                |
| Actel IGLOO [3], ProASIC [4; 5]            | Fine        | Three-input logic function                    | Horizontal and vertical tracks | 4-Kbit blocks or 1 K FlashROM         | Flash-based, low-power mode                                             |
| Altera Stratix III [15], Stratix IV [16]   | Fine        | Eight-input adaptive logic module             | Horizontal and vertical tracks | 640-bit, 9-Kbit, 144-Kbit blocks      | DSP blocks, 1.6–8.5 Gbps I/Os programmable power                        |
| Xilinx Virtex II Pro [262], Virtex 4 [263] | Fine        | Four-input LUTs                               | Horizontal and vertical tracks | 18-Kbit blocks                        | Block multipliers, DSP blocks, PowerPC 405 processor, 3.1–6.5 Gbps I/Os |
| Xilinx Virtex 5 [264], Virtex 6 [265]      | Fine        | Six-input LUTs or dual five-input             | Horizontal and vertical tracks | 36-Kbit blocks                        | DSPs, 6.5 Gbps I/Os PowerPC 440 processor                               |
| Lattice XP2 [149]                          | Fine        | Four-input LUTs                               | Horizontal and vertical tracks | 18-Kbit blocks                        | DSPs, internal flash memory                                             |
| SiliconBlue iCE65 [219]                    | Fine        | Four-input LUTs                               | Horizontal and vertical tracks | 4-Kbit blocks                         | Low power mode, internal flash memory                                   |
| Silicon Hive Avispa [220]                  | Coarse      | ALUs, shifters, accumulators, and multipliers | Buses                          | 4 × 4 K dual port local data memories | 75 FUs, 95 register files, OFDM radio applications                      |

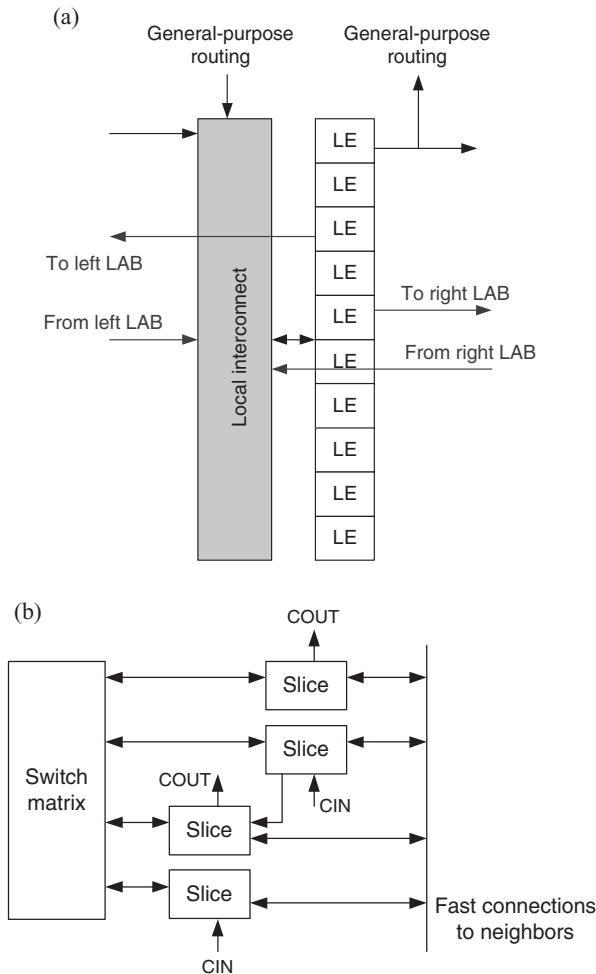
OFDM: orthogonal frequency-division multiplexing.

or a small number of bits. The most common kind of fine-grained FUs are the small LUTs that are used to implement the bulk of the logic in a commercial FPGA. A coarse-grained FU, on the other hand, is typically much larger, and may consist of arithmetic and logic units (ALUs) and possibly even a significant amount of storage. In this section, we describe the two types of FUs in more detail.

Many reconfigurable systems use commercial FPGAs as a reconfigurable fabric. These commercial FPGAs contain many three to six input LUTs, each of which can be thought of as a fine-grained FU. Figure 6.3a illustrates a LUT; by shifting in the correct pattern of bits, this FU can implement any single



**Figure 6.3** Fine-grained reconfigurable FUs [244]. (a) Three-input LUT; (b) cluster of LUTs.



**Figure 6.4** Commercial logic block architectures. (a) Altera LAB [14]; (b) Xilinx CLB [262].

function of up to three inputs—the extension to LUTs with larger numbers of inputs is clear. Typically, LUTs are combined into clusters, as shown in Figure 6.3b. Figure 6.4 shows clusters in two popular FPGA families. Figure 6.4a shows a cluster in the Altera Stratix device; Altera calls these clusters “logic array blocks” (LABs) [14]. Figure 6.4b shows a cluster in the Xilinx architecture [262]; Xilinx calls these clusters “configurable logic blocks” (CLBs). In the Altera diagram, each block labeled “LE” is an LUT, while in the Xilinx diagram, each “Slice” contains two LUTs.

Reconfigurable fabrics containing LUTs are flexible and can be used to implement any digital circuit. However, compared to the coarse-grained structures described below, these fine-grained structures have significantly more

area, delay, and power overhead. Recognizing that these fabrics are often used for arithmetic purposes, FPGA companies have included additional features such as carry chains and cascade chains to reduce the overhead when implementing common arithmetic and logic functions.

While the efficiency of commercial FPGAs is improved by adding architectural support for common functions, one can go further and embed significantly larger, but less flexible, reconfigurable FUs. There are two kinds of devices that contain coarse-grained FUs.

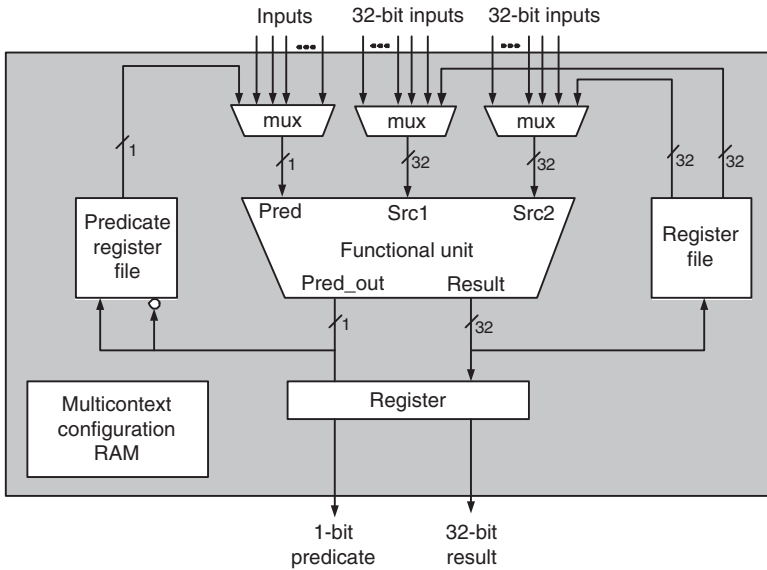
First, many commercial FPGAs, which consist primarily of fine-grained FUs, are increasingly enhanced by the inclusion of larger blocks. For instance, the early Xilinx Virtex device contains embedded  $18 \times 18$  bit multiplier units [262]. When implementing algorithms requiring a large amount of multiplication, these embedded units can significantly improve the density, speed, and power consumption. On the other hand, for algorithms that do not perform multiplication, these blocks are rarely useful. The Altera Stratix devices contain a larger, but more flexible embedded block, called a DSP block, which can perform accumulate functions as well as multiply operations. The comparison between the two devices clearly illustrates the flexibility and overhead trade-off: the Altera DSP block may be more flexible than the Xilinx multiplier, but it consumes more chip area and runs slower for the specific task of multiplication. Recent Xilinx devices have a more complex embedded unit, called DSP48. It should be noted that, while such embedded blocks eliminate reconfigurable interconnects within them, their fixed location can cause wiring congestion and overhead. Moreover, they would become an overhead for applications that do not make use of them.

Second, while commercial FPGAs described above contain both fine-grained and coarse-grained blocks, there are also devices that contain only coarse-grained blocks. An example of a coarse-grained architecture is the ADRES architecture, which is shown in Figure 6.5 [171]. Each reconfigurable FU in this device contains a 32-bit ALU that can be configured to implement one of several functions including addition, multiplication, and logic functions, with two small register files. Clearly, such an FU is far less flexible than the fine-grained FUs described earlier; however, if the application requires functions that match the capabilities of the ALU, these functions can be efficiently implemented in this architecture.

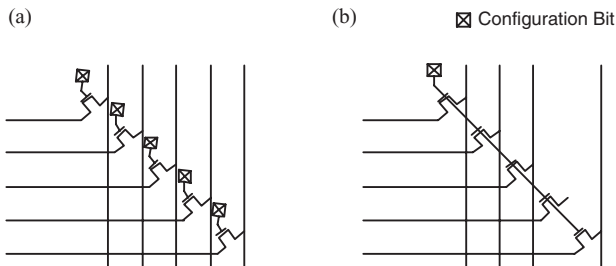
## 6.5.2 Reconfigurable Interconnects

Regardless of whether a device contains fine-grained FUs, coarse-grained FUs, or a mixture of the two, the FUs needed to be connected in a flexible way. Again, there is a trade-off between the flexibility of the interconnect (and hence the reconfigurable fabric) and the speed, area, and power efficiency of the architecture.

Reconfigurable interconnect architectures can be classified as fine grained or coarse grained. The distinction is based on the granularity with which wires are switched. This is illustrated in Figure 6.6, which shows a flexible intercon-



**Figure 6.5** ADRES reconfigurable FU [171]. Pred is a one-bit control input selecting either Src1 or Src2 for the functional unit.

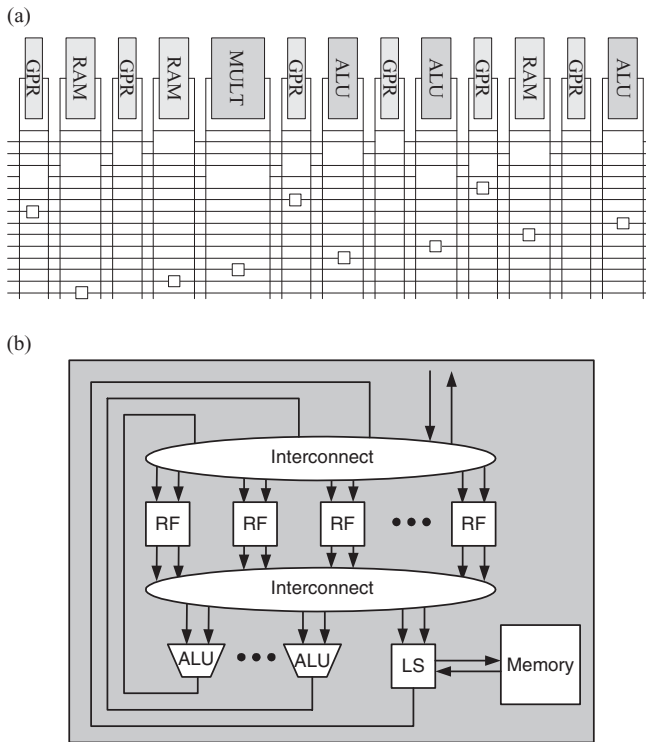


**Figure 6.6** Routing architectures. (a) Fine grained; (b) coarse grained.

nect between two buses. In the fine-grained architecture in Figure 6.6a, each wire can be switched independently, while in Figure 6.6b, the entire bus is switched as a unit. The fine-grained routing architecture in Figure 6.6a is more flexible, since not every bit needs to be routed in the same way; however, the coarse-grained architecture in Figure 6.6b contains fewer programming bits, and hence has lower overhead.

Fine-grained routing architectures are usually found in commercial FPGAs. In these devices, the FUs are typically arranged in a grid pattern and they are connected using horizontal and vertical channels. Significant research has been performed in the optimization of the topology of this interconnect [157].

Coarse-grained routing architectures are commonly used in devices containing coarse-grained FUs. Figure 6.7 shows two examples of coarse-grained routing architectures. The routing architecture in Figure 6.7a is used in the



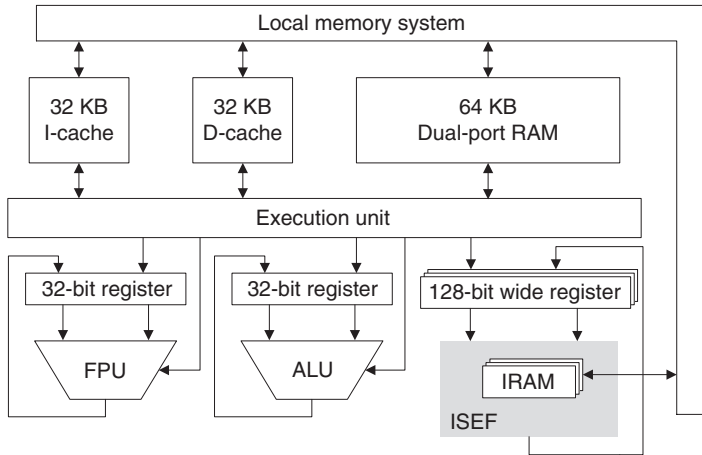
**Figure 6.7** Examples of coarse-grained routing architectures. (a) Totem coarse-grained routing architecture [60]; (b) Silicon Hive coarse-grained routing architecture [220]. GPR: General Purpose Registers, MULT: Multiplier. RF: register file, LS: Load/Store Unit.

Totem reconfigurable system [60]; the interconnect is designed to be flexible and to provide arbitrary connection patterns between FUs. On the other hand, the routing architecture in Figure 6.7b, which is used in the Silicon Hive reconfigurable system, is less flexible, but faster and smaller [220]. In the Silicon Hive architecture, only connections between units that are likely to communicate are provided.

### 6.5.3 Software Configurable Processors

Software configurable processors are devices introduced by Stretch. They have an architecture that couples a conventional instruction processor to a reconfigurable fabric to allow application programs to dynamically customize the instruction set. Such architectures have two benefits. First, they offer significant performance gains by exploiting data parallelism, operator specialization, and deep pipelines. Second, application builders can develop their programs using the Stretch C compiler without having expertise in electronic design.





**Figure 6.8** The Stretch S6 Software Configurable Processor Engine [230]. IRAM denotes embedded memory for the instruction set extension fabric (ISEF).

A software configurable processor consists of a conventional 32-bit RISC processor coupled with a programmable instruction set extension fabric (ISEF). There are also an ALU for arithmetic and logic operations and a floating-point unit (FPU) for floating-point operations. Figure 6.8 shows the S6 Software Configurable Processor Engine.

The ISEF consists of an array of blocks, each containing an array of 4-bit ALUs and an array of multiplier elements, interconnected by a programmable routing fabric. The 4-bit ALUs can be cascaded through a fast carry circuit to form up to 64-bit ALUs. Each 4-bit ALU may also implement up to four 3-input logic functions, with four register bits for extension instruction state variables or for pipelining.

The ISEF supports multiple application-specific instructions as extension instructions. Arguments to extension instructions are provided from 32 wide registers, which are 128 bits wide. Each extension instruction may read up to three 128-bit operands and write up to two 128-bit results. A rich set of dedicated load and store instructions are provided to move data between the 128-bit wide register and the 128-bit wide cache and memory subsystem. The ISEF supports deep pipelining by allowing extension instructions to be pipelined.

In addition to the load/store model, a group of extension instructions may also define arbitrary state variables to be held in registers within the ISEF. State values may be read and modified by any extension instruction in the group, thereby reducing the Wide Register traffic.

In addition to the Software Configurable Processor Engine, there is also a programmable accelerator, which consists of a list of functions implemented in dedicated hardware. These functions include motion estimation for video encoding, entropy coding for H.264 video, cryptographic operations based on

Advanced Encryption Standard (AES) and Triple Data Encryption Standard (3DES) schemes, and various audio codecs including those for MP3 and AC3.

To develop an application, the programmer identifies critical sections to be accelerated, writes one or more extension instructions as functions in a variant of the C programming language called Stretch C, and accesses those functions from the application program. Further information about application mapping for software configurable processors can be found in the next section, and related application studies for the Stretch S5 Software Configurable Processor Engine can be found in Sections 7.6.2 and 7.7.2.

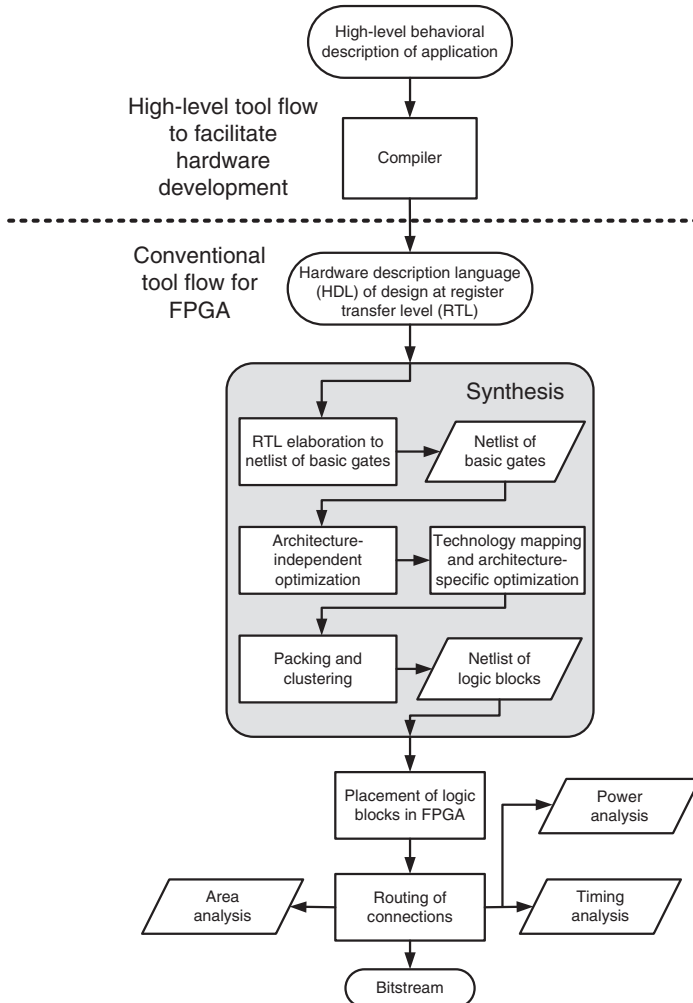
## 6.6 MAPPING DESIGNS ONTO RECONFIGURABLE DEVICES

The resources in a reconfigurable device need to be configured appropriately to implement a design for a given application. We shall look at the ways designs are mapped onto an FPGA and onto a software configurable processor.

A typical tool flow for an FPGA is shown in Figure 6.9 [56]. In the conventional tool flow, HDLs such as VHDL and Verilog are widely used to target commercial devices to describe the circuit to be implemented in the FPGA. The description of the circuit is written at the register transfer level (RTL), which specifies the operations at each clock cycle. The description is then synthesized to a netlist of logic blocks before being placed and routed for the FPGA.

In the first stage of the synthesis process, the datapath operations in an RTL design such as control logic, memory blocks, registers, adders, and multipliers are identified and elaborated into a set of basic boolean logic gates such as AND, OR, and XOR.

Next, the netlist of basic gates is optimized independent of the FPGA architecture. The optimization includes: boolean expression minimization, removing the redundant logic, buffering sharing, retiming, and finite-state machine encoding. The optimized netlist of basic gates is then mapped to the specific FPGA architecture such as Xilinx Virtex devices or Altera Stratix devices. There is further optimization based on the specific architecture such as carry chains for adders and dedicated shift functions in logic block for shift registers. The final stage in the synthesis process is packing and clustering groups of several LUTs and registers into logic blocks like Figure 6.4. The packing and clustering minimize the number of connections between different logic blocks. After the synthesis process, the logic blocks in the mapped netlist are placed onto the FPGA based on the different optimization goals, such as circuit speed, routability, and wire length. Once the location of the logic blocks is determined, the connection between I/Os, logic blocks, and other embedded elements are routed onto the programmable routing resources in FPGA. The routing process determines which programmable switches should be used to connect the logic block input and output pins. Finally, a configuration bitstream for all inputs and outputs, logic blocks, and routing resources for the circuit in specific FPGA is generated.



**Figure 6.9** Tool flow for FPGA.

The above description covers design mapping for fine-grained FPGA resources. Many reconfigurable devices also have coarse-grained resources for computation and for storage (see Section 6.5.1), so FPGA design tools need to take such resources into account.

To improve productivity, high-level programming languages are included in the tool flow for FPGAs (the upper part of Figure 6.9). These languages and tools, such as AutoPilot [272], Harmonic [159], and ROCC [248], enable application developers to produce designs without detailed knowledge of the implementation technology. Some of these compilers are able to extract parallelism in the computation from the source code, and to optimize for pipelining. Such tools often improve productivity of application developers, at the expense

of the quality of the design. However, since device capacity and capability continue to increase rapidly, productivity of application developers is likely to become the highest priority.

Besides configuring the circuit, there are tools that analyze the delay, area, and power consumption of the implemented circuit. These tools are used to check whether the circuit meets the requirements of the application developer.

For a Software Configurable Processor from Stretch, the compilation needs to target both the execution unit and the ISEF shown in Figure 6.8. The first stage of the Stretch C compiler [25] takes an extension instruction and applies various optimizations, such as constant propagation, loop unrolling, and word-length optimization. In addition, sequences of operators are aggregated into balanced trees where possible; operators are specialized with multiplication by a constant converted into shifts and adds; and resources among operators for different instructions are shared. The compiler then produces two main outputs: instruction header and latency information including register usage for the Xtensa compiler, and a structured netlist of the operators extracted from the source code for mapping to the resources in the ISEF [25].

The Xtensa compiler compiles the application code with references to the extension instructions. It uses the instruction header and timing data from the Stretch C compiler to perform register allocation and optimized scheduling of the instruction stream. The result is then linked to the ISEF bitstream.

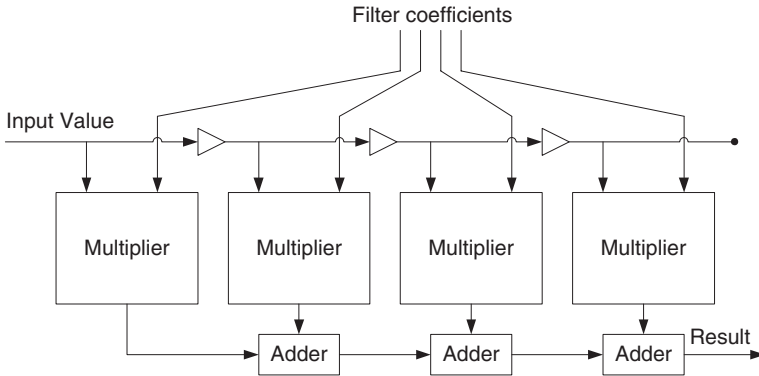
The ISEF bitstream generation stage in the Stretch compiler is similar to the tool flow for FPGAs shown earlier. The four main stages are [25]:

- *Map*. Performs module generation for the operators provided by the initial stage of the Stretch C compiler.
- *Place*. Assigns location to the modules generated by Map.
- *Route*. Performs detailed, timing-driven routing on the placed netlist.
- *Retime*. Moves registers to balance pipeline stage delays.

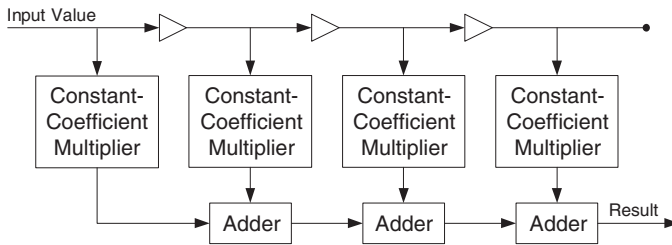
The linker packages the components of the application into a single executable file, which contains a directory of the ISEF configurations for the operating system or the run-time system to locate instruction groups for dynamic reconfiguration.

## 6.7 INSTANCE-SPECIFIC DESIGN

Instance-specific design is often used in customizing both hardware and software. The aim for instance-specific design is to optimize an implementation for a particular computation. The main benefits are improving speed and reducing resource usage, leading to lower power and energy consumption at the expense of flexibility.



**Figure 6.10** An FIR filter containing two-input multipliers that support variable filter coefficients. The triangular blocks denote registers [197].



**Figure 6.11** An FIR filter containing one-input constant-coefficient multipliers that support instance-specific filter coefficients [197].

We describe three techniques for automating instance-specific design. The first technique is constant folding: propagating static input values through a computation to eliminate unnecessary hardware or software. As an example, an instance-specific version of the hardware design in Figure 6.10 specialized to particular filter coefficients is shown in Figure 6.11. The improvement in efficiency is due to the fact that one-input constant-coefficient multipliers are smaller and faster than two-input multipliers.

The ability to implement specialized designs, while at the same time providing flexibility by allowing different specialized designs to be loaded onto a device, can allow reconfigurable logic to be more effective than ASICs in implementing some applications. For other applications, performance improvements from optimizing designs to a particular problem instance can help to shift the price/performance ratio away from ASICs and toward FPGAs.

Significant benefits for instance-specific design have been reported for a variety of applications. For FIR (Finite Impulse Response) filters, a modified common subexpression elimination algorithm can be used to reduce the number of adders used in implementing constant-coefficient multiplication [180]. Up to 50% reduction in the number of FPGA slices and up to 75%

reduction in the number of LUTs for fully parallel implementations have been observed, in comparison to designs based on distributed arithmetic. Moreover, there is up to 50% reduction in the total dynamic power consumption of the FIR filters.

Changing an instance-specific design at run time is usually much slower than changing the inputs of a general circuit, since a new full or partial configuration must be loaded, which may take many tens or hundreds of milliseconds. It is therefore important to carefully choose how a design is specialized. Related discussions on run-time reconfiguration can be found in Section 6.9.

The second technique for automating instance-specific design is function adaptation, which involves changing a function in hardware or software to achieve, for a specific application instance, the desired trade-off in the performance or resource usage of the function and the quality of the result produced by the function.

An example of function adaptation is word-length optimization. Given the flexibility of fine-grain FPGA, it is desirable to automate the process of finding a good custom data representation. An important implementation decision to automate is the selection of an appropriate word length and scaling for each signal in a DSP system. Unlike microprocessor-based implementations where the word length is defined a priori by the hard-wired architecture of the processor, reconfigurable computing based on FPGAs allows the size of each variable to be customized to produce the best trade-offs in numerical accuracy, design size, speed, and power consumption.

The third technique for automating instance-specific design is architecture adaptation, which involves changing the hardware and software architecture to optimize for a specific application instance, such as supporting relevant custom instructions. We shall discuss this technique in more detail in the next section.

An illustration of the above three techniques is given in Table 6.3. Further information about instance-specific design can be found elsewhere [197].

**TABLE 6.3 Some Illustrations of Instance-Specific Design**

| Technique               | Purpose                                        | Example                                  | Benefits in Example                              |
|-------------------------|------------------------------------------------|------------------------------------------|--------------------------------------------------|
| Constant folding        | Optimize operation for static input values     | FIR filter [180]                         | Up to 50% reduction in dynamic power consumption |
| Function adaptation     | Optimize function for quality of result        | Word-length optimization [62]            | 87% reduction in power consumption               |
| Architecture adaptation | Optimize architecture for application instance | Instruction processor customization [77] | Speed improved by 72%, area increased by 3%      |

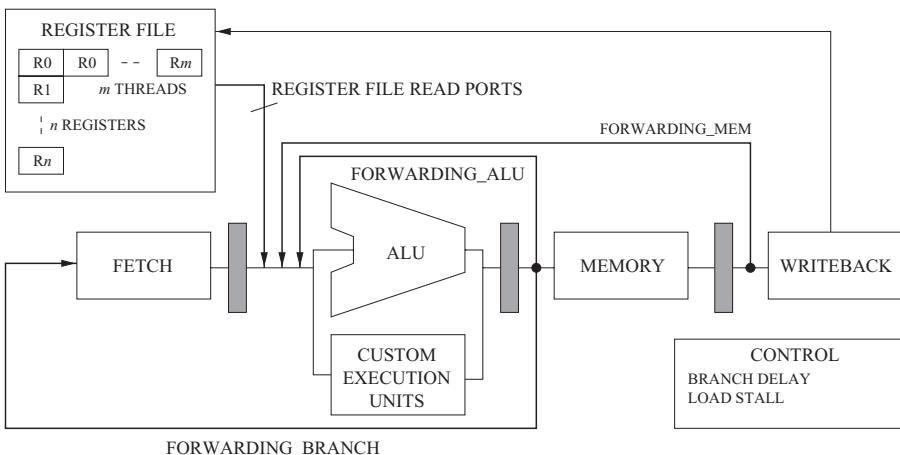
## 6.8 CUSTOMIZABLE SOFT PROCESSOR: AN EXAMPLE

This section describes a multithreaded soft processor called CUSTARD with a customizable instruction set [77]. It illustrates the material from the preceding sections: we show how an instruction processor can be customized by adapting the architecture to support different types of multithreading and custom instructions; we then present the associated tool flow targeting reconfigurable technology.

CUSTARD supports multiple contexts within the same processor hardware. A context is the state of a thread of execution, specifically the state of the registers, stack, and program counter. Supporting threads at the hardware level brings two significant benefits. First, a context switch—changing the active thread—can be accomplished within a single cycle, enabling a uniprocessor to interleave execution of independent threads with little or no overhead. Second, a context switch can be used to hide latency where a single thread would otherwise busy-wait.

The major cost of supporting multiple threads stems from the additional register files required for each context. Fortunately, current FPGAs are rich in on-chip block memories that could be used to implement large register files. Additional logic complexity must also be added to the control of the processor and the current thread must be recorded at each pipeline stage. However, the bulk of the pipeline and the FUs are effectively shared between multiple threads, so we would expect a significant area reduction over a multiprocessor configuration.

Instances of CUSTARD processors are generated using a parameterizable model. The key elements of this parameterizable model are shown in Figure 6.12. This model is used both in instantiating a synthesizable hardware description and in configuring a cycle-accurate simulator.



**Figure 6.12** CUSTARD microarchitecture showing threading, register file, and forwarding network parameterizations.

The CUSTARD base architecture is typical of a soft processor, with a fully bypassed and interlocked 4-stage pipeline. It is a load/store RISC architecture supporting the MIPS integer instruction set. It is also capable of augmenting the pipeline with custom instructions using spare portions of the MIPS opcode space.

There are four sets of parameters for customizing CUSTARD. The first set covers multithreading support: one can specify the number of threads and the threading type, either block multithreading (BMT) or interleaved multithreading (IMT). The second set covers custom instructions and the associated datapaths at the execution stage of the pipeline as well as custom memory blocks. The third set covers the forwarding and interlock architecture: whether the Branch delay slot, the Load delay slot, and the forwarding paths are necessary. The fourth set covers the register file: the number of registers and the number of register file ports.

Two types of multithreading, BMT and IMT, are supported. Both types simultaneously maintain the context—the state of registers, program counter, and so on—of multiple independent threads. The types of threading differ in the circumstances that context switches are triggered.

BMT triggers a context switch as a result of some run-time event in the currently active thread, for example, a cache miss, an explicit “yield” of control, or the start of some long latency operation such as a custom instruction. When only a single thread is available, the BMT processor behaves exactly as a conventional single-threaded processor. When multiple threads are available, any latency in the active thread is hidden by a context switch. The context switch is triggered at the execution stage of the pipeline, such that the last instruction fetched must be flushed and refilled from the new active thread.

IMT performs a mandatory context switch every single cycle, resulting in interleaved execution of the available threads. IMT permits simplification of the processor pipeline since, given sufficient threads, certain pipeline stages are guaranteed to contain independent instructions. IMT thus removes pipeline hazards and permits simplification of the forwarding and interlock network designed to mitigate these hazards. The CUSTARD processor can exploit this capability by selectively removing forwarding paths to optimize the processor for a particular threading configuration.

Table 6.4 summarizes the customization of the forwarding and interlock architecture for each multithreading configuration. The forwarding paths, BRANCH, ALU, and MEM, are illustrated in Figure 6.12. The IMT columns show how elements of the forwarding and interlock network can be removed depending upon the number of available threads. For example, in the case of two threads, the ALU forwarding logic can be removed. When two IMT threads are available, any instruction entering the ALU stage of the pipeline is independent of the instruction leaving the ALU stage. Removing interlocks in situations highlighted by “\*” constrains the ordering of the input instructions; the relevant parameters are made available to the compiler, which can then adapt the scheduling of instructions.



**TABLE 6.4 Summary of Forwarding Paths (As Shown in Figure 5.12) and Interlocks That Can Be “Optimized Away” for Single-Threaded, Block Multithreaded (BMT), and Interleaved Multithreaded (IMT) Parameterizations**

| Disable           | Configuration Number of Threads | BMT $\geq 1$ | IMT 2 | IMT $\geq 4$ |
|-------------------|---------------------------------|--------------|-------|--------------|
| FORWARDING BRANCH |                                 |              | ✓     | ✓            |
| FORWARDING ALU    |                                 | ✓            | ✓     |              |
| FORWARDING MEM    |                                 |              | ✓     |              |
| BRANCH DELAY      |                                 | ✓*           | ✓     | ✓            |
| LOAD INTERLOCK    |                                 | ✓*           | ✓     | ✓            |

\*Optimizing away this element in this configuration changes the compiler scheduler behavior to prevent hazards.

Multiple contexts are supported by multiple register files that are implemented as dual-port RAM on the FPGA. Each register file access is indexed by the register number and also the ID of the thread that generated the access. Each register file is also parameterizable in terms of the number of ports and the number of registers per thread. Increasing the number of register file ports allows custom instructions to be selected by the compiler that take a greater number of operands.

An approach based on compiling a parallel imperative language into hardware [191] is used to implement the parameterizable processor. This implementation of CUSTARD provides a framework for parameterization of the processor together with a route to hardware. The associated compiler outputs MIPS integer instructions and custom instructions to optimize CUSTARD for a given application; Table 6.5 shows the custom instructions for some benchmarks.

Figure 6.13 shows the flow through the CUSTARD tools. Custom instructions are generated based on a technique known as similar sub-instructions [77]. Prior to finding custom instructions, a preoptimization stage performs standard source-level optimizations together with loop unrolling to expose loop parallelism. After custom instructions have been selected, custom and base instructions are scheduled to minimize pipeline stalls. This scheduling stage is parameterizable to support the microarchitectural changes afforded by the CUSTARD multithreading modes.

The result of compilation comprises hardware datapaths to implement custom instructions and software to execute on the customized processor. Custom instruction datapaths are added to the CUSTARD processor, and the decoding logic is revised to map new instructions to unused portions of the opcode space.

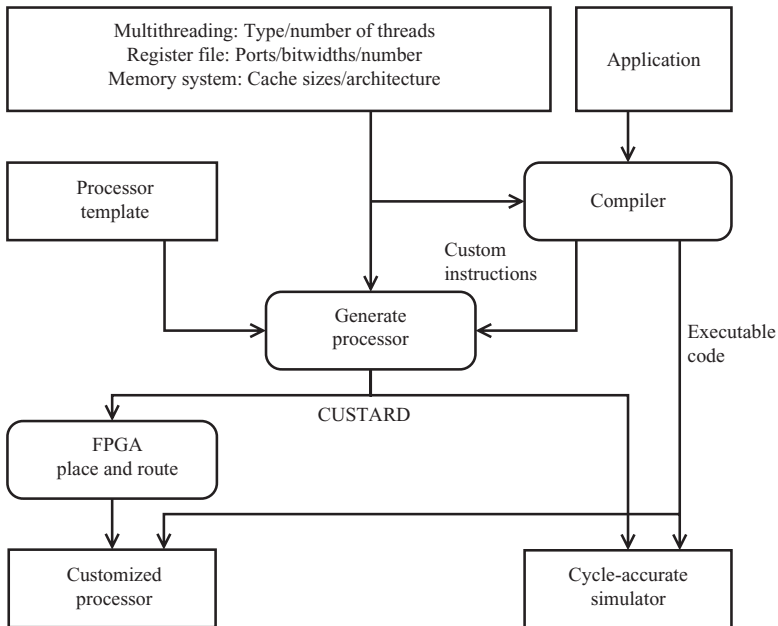
There is a cycle-accurate simulator based upon the SimpleScalar framework [30]. The simulator can be configured directly from the processor hardware description and simulates a parameterizable memory system.

Five benchmarks—Blowfish, Colourspace, AES, discrete cosine transform (DCT), and SUSAN—have been developed for a CUSTARD processor

**TABLE 6.5 A Summary of the Custom Instructions Automatically Generated for a Set of Benchmarks**

| Benchmark   | Custom Instruction(s)<br>(Input Registers $r_0 - r_3$ ,<br>Immediate Value $imm_0$ ) | Num.<br>Uses | Latency<br>(Cycles) | BRAM<br>(Bytes) |
|-------------|--------------------------------------------------------------------------------------|--------------|---------------------|-----------------|
| Blowfish    | $LUT(r_0 \gg 24) + LUT(r_1 \gg 16)$                                                  | 2            | 1                   | 1024            |
|             | $LUT([r_0 \gg 8] \& 255)$                                                            | 2            | 1                   |                 |
| Color space | $([r_0 \gg 8] \& 0xF F)   (r_1 \& 0xF F 00)$<br>$  ([r_2 \ll 8] \& 0xF F 0000)$      | 1            | 1                   | 32              |
| DCT         | $LUT(r_1) + r_2 * (r_0 \ll 8)$                                                       | 65           | 2                   | 64              |
|             | $LUT(r_1) + r_2 * ([r_0 \& 255] - 128)$                                              | 65           | 2                   |                 |
| Edge detect | $LUT(r_0 + 1 + imm_0)$                                                               | 3            | 1                   | 64              |
| Susan       | $LUT(r_0)$                                                                           | 31           | 1                   | 516             |
| AES         | $LUT(r_0) \wedge LUT(r_1 \gg 8)$                                                     | 64           | 1                   | 1024            |
|             | $\wedge LUT(r_2 \gg 16) \wedge LUT(r_3 \gg 24)$                                      |              |                     |                 |

Inputs  $r_0 - r_3$  are allocated to registers from the general purpose file.  $LUT(a)$  = table lookup from dedicated block RAM (BRAM) address  $a$ . “Num. Uses” demonstrates the extent of reuse by showing the number of times the instruction is used in the benchmark assembly code. Latency is the number of execution cycles required before the output is available to the forwarding network or in the register file.

**Figure 6.13** Tool flow for the CUSTARD processor customized for a particular application.

implemented on a Xilinx XC2V2000 FPGA. It is found that the IMT4 (IMT with four threads) and BMT4 (BMT with four threads) configurations add only 28% and 40% area, respectively, to the single-threaded processor, while allowing interleaved execution of four threads with no software overhead. Moreover, custom instructions give a significant performance increase, an average of 72% with a small area overhead above the same configuration without custom instructions, an average of only 3%. CUSTARD accelerates AES by 355%.

The IMT processors without custom instructions provide a higher maximum clock rate than both BMT (41% higher) and single-threaded (5% higher) processors. The number of cycles is also reduced by an average of 10%. The IMT processors hide pipeline latencies by tightly interleaving independent threads. We anticipate that the relatively low (10%) improvement is caused by the short latency of the custom instructions generated (Table 6.5), at most two cycles in every case. It is not possible to build longer latency instructions within the register file port constraints, so we expect that deeply pipelined processors or floating point custom instructions are needed to create latencies long enough for significant benefit in this area. However, the IMT processors allow a higher maximum clock rate by removing the forwarding logic around the ALU. The ALU forwarding logic is necessarily on the critical path in the BMT and single-threaded processors, as indicated by the timing analyzer reports.

## 6.9 RECONFIGURATION

There are many motivations for reconfiguration. One motivation is to share resources that are not required concurrently. Another motivation is to upgrade to support new functions, new standards, or new protocols. A third reason is to adapt the hardware based on run-time conditions.

Run-time reconfiguration has shown promise for many applications, including automotive systems [35], high-performance computing [81], interconnection networks [161], video processing [211], and adaptive Viterbi decoding [241]. The treatment below abstracts from the specific technology, focusing on overhead analysis of designs involving run-time reconfiguration.

### 6.9.1 Reconfiguration Overhead Analysis

Adapting an architecture to specific applications aims to improve performance by applying application-specific optimizations. However the performance gained by adaptation has to outweigh the cost of reconfiguration. Consider a software function  $f()$  that takes  $t_{si}$  time to execute on an architecture with a standard, general-purpose instruction set. When adapted to support a custom instruction, it takes  $t_{ci}$  time to execute, and can be expressed as a fraction of the original execution time as  $\alpha t_{si}$ , where  $0 < \alpha < 1$ . The device on which  $f()$

executes requires  $t_r$  time to reconfigure. We define a reconfiguration ratio  $R$  that helps us decide if reconfiguration to this new architecture is desirable by analyzing the overhead involved:

$$R = \frac{t_{si}}{t_{ci} + t_r} = \frac{t_{si}}{\alpha t_{si} + t_r}. \quad (6.1)$$

The reconfiguration ratio  $R$  gives us a measure of the benefits of reconfiguring to the new adapted architecture;  $R$  provides the improvement factor of the new implementation after reconfiguration, over the current implementation. The point where  $R = 1$  is the threshold: if  $R > 1$ , reconfiguration will be beneficial. It is, however, important to note that an  $R$  value of 2 does not necessarily translate to an overall twofold system performance increase. The maximum reconfiguration ratio  $R_{max}$  is a measure of the absolute performance gain, discounting reconfiguration time. It is the maximum possible  $R$  value for a custom instruction. This is determined as follows:

$$R_{max} = \lim_{t_r \rightarrow 0} R = \frac{t_{si}}{t_{ci}} = \frac{1}{\alpha}. \quad (6.2)$$

The maximum potential reconfiguration ratio  $R_{pot}$  is a measure of the maximum rate at which performance improvement is possible; it gives an idea of how quickly a custom instruction will cross the reconfiguration threshold:

$$R_{pot} = \lim_{\alpha \rightarrow 0} R = \frac{t_{si}}{t_r}. \quad (6.3)$$

$R_{pot}$  provides an indication on the granularity of reconfiguration and size of functions that will benefit from adaptation. A function with a higher  $R_{pot}$  value can be adapted more easily. It can be shown that:

$$\frac{t_{ci}}{t_{si}} = \frac{R_{pot} - 1}{R_{pot}}. \quad (6.4)$$

Hence an  $R_{pot}$  of 5 means that the custom instruction produced need only be four-fifths the speed of the original software version, before it becomes worth implementing. An  $R_{pot}$  of 4 requires the custom instruction to be three-fourths the speed of the original, while an  $R_{pot}$  of 3, two-thirds the speed of the original, and so on.

If function  $f()$  has an  $R_{pot}$  value of 1, the adaptation and subsequent reconfiguration of that function will not be beneficial.

The reconfiguration ratio can also be described in terms of the number of clock cycles of an FIP. Consider a software function  $f()$ , which takes  $t_{si}$  units of time to execute. The function takes  $n_{si}$  clock cycles to execute, and the cycle

time is  $T_{si}$ . The function  $f()$  is called  $F$  times over the period we are investigating, in this case one execution of the application. Similarly with  $t_{ci}$ :

$$\begin{aligned} t_{si} &= n_{si}T_{si}F \\ t_{ci} &= n_{ci}T_{ci}F. \end{aligned} \quad (6.5)$$

The reconfiguration time  $t_r$  can be rewritten as the product of the reconfiguration cycle time  $T_r$  and the number of configuration cycles  $n_r$ . The reconfiguration cycle time,  $T_r$ , is platform dependent and is independent of the cycle time of designs implemented on the programmable device. The number of configuration cycles models the time required for either full or partial reconfiguration. In full reconfiguration,  $n_r$  is a constant associated with a particular programmable device; in partial reconfiguration,  $n_r$  varies with the amount of changes in the configuration.  $n_r$  may also be reduced through improvements in technology and architectures that support fast reconfiguration through caches or context switches. There is a factor  $\tau$  that represents certain reconfiguration overheads, such as stopping of the device prior to reconfiguration and starting of the device after reconfiguration:

$$t_r = n_rT_r + \tau. \quad (6.6)$$

In modern programmable devices, the time taken to start and stop a device can often be ignored. For instance, in Xilinx Virtex devices, this value can be as small as 10% of the time required to reconfigure a frame, the smallest atomic reconfigurable unit.

Other overheads include the time taken to save and restore the state of the processor. In the most extreme case, the state of the processor includes all storage components in the processor, for instance the register file, pipeline registers, program counter, or cache. By reinstating the state of a processor, a processor can be put back into the condition it was in when the state was saved.

After substituting Equations 6.5 and 6.6 into Equation 6.1, the reconfiguration ratio  $R$  becomes:

$$R = \frac{t_{si}}{t_{ci} + t_r} = \frac{n_{si}T_{si}F}{n_{ci}T_{ci}F + n_rT_r + \tau}. \quad (6.7)$$

This equation can be used to produce a graph showing how  $R$  changes with increasing  $F$ , the number of times that the function  $f()$  is called. When  $R > 1$ , then reconfiguration is profitable. More information about this approach can be found in a description of the adaptive flexible instruction processor [213].

### 6.9.2 Trade-Off Analysis: Reconfigurable Parallelism

In the following we describe a simple analytical model [37] for devices that are partially reconfigurable: the larger the configured area, the longer the

configuration time. For applications involving repeated independent processing of the same task on units of data that can be supplied sequentially or concurrently, increasing parallelism reduces processing time but increases configuration time. The model below would help identify the optimal trade-off: the amount of parallelism that would result in the fastest overall combination of processing time and configuration time.

The three implementation attributes are performance, area, and storage: processing time  $t_p$  for one unit of data, with area  $A$ , configuration time  $t_r$ , and configuration storage size  $\Psi$ . There are  $s$  processing steps, and the amount of parallelism is  $P$ .

We can also identify parameters of the application: the required data throughput is  $\phi_{app}$ , while there are  $n$  units of data  $n$  processed between successive configurations. The reconfigurable device has available area  $A_{max}$ . The data throughput of the configuration interface is  $\phi_{config}$ .

Designs on volatile FPGAs require external storage for the initial configuration bitstream. Designs using partial run-time reconfiguration also need additional storage for the precompiled configuration bitstreams of the reconfigurable modules. Given  $A$  denotes the size of a reconfigurable module in FPGA tiles (e.g., CLBs) and  $\Theta$  denotes a device-specific parameter that specifies the number of bytes required to configure one tile, the partial bitstream size and storage requirement  $\Psi$  (in bytes) of a reconfigurable module is directly related to its area  $A$ :

$$\Psi = A \cdot \Theta + h \approx A \cdot \Theta, \quad (6.8)$$

where  $h$  denotes the header of configuration bitstreams. In most cases, this can be neglected because the header size is very small.

The time overhead of run-time reconfiguration can consist of multiple components, such as scheduling, context save and restore, as well as the configuration process itself. In our case there is no scheduling overhead as modules are loaded directly as needed. There is also no context that needs to be saved or restored since signal processing components do not contain a meaningful state once a dataset has passed through. The reconfiguration time is proportional to the size of the partial bitstream and can be calculated as follows:

$$t_r = \frac{\Psi}{\phi_{config}} \approx \frac{A \cdot \Theta}{\phi_{config}}. \quad (6.9)$$

$\phi_{config}$  is the configuration data rate and measured in *bytes per second*. This parameter not only depends on the native speed of the configuration interface but also on the configuration controller and the data rate of the memory where the configuration data are stored.

We can distinguish between run-time reconfigurable scenarios where data do not have to be buffered during reconfiguration and scenarios where data

**TABLE 6.6 Buffer Size for Various Functions and Reconfiguration Times**

| Function                 | Data Throughput | Buffer Size for a Given Reconfiguration Time |           |           |
|--------------------------|-----------------|----------------------------------------------|-----------|-----------|
|                          |                 | 100 ms                                       | 10 ms     | 1 ms      |
| Downconversion (16 bits) | 800 Mbit/s      | 80 Mbit                                      | 8 Mbit    | 800 Kbit  |
| Downconversion (14 bits) | 700 Mbit/s      | 70 Mbit                                      | 7 Mbit    | 700 Kbit  |
| Demodulation UMTS        | 107.52 Mbit/s   | 10.75 Mbit                                   | 1.07 Mbit | 107 Kbit  |
| Demodulation GSM         | 7.58 Mbit/s     | 758 Kbit                                     | 75.8 Kbit | 7.58 Kbit |
| Error correction UMTS    | 6 Mbit/s        | 600 Kbit                                     | 60 Kbit   | 6 Kbit    |
| Error correction GSM     | 22.8 Kbit/s     | 2.28 Kbit                                    | 228 bit   | 22.8 bit  |
| Decryption UMTS          | 2 Mbits/s       | 200 Kbit                                     | 20 Kbit   | 2 Kbit    |
| Encryption GSM           | 13 Kbit/s       | 1.3 Kbit                                     | 130 bit   | 13 bit    |

UMTS: Universal Mobile Telecommunications System, GSM: Global System for Mobile Communications

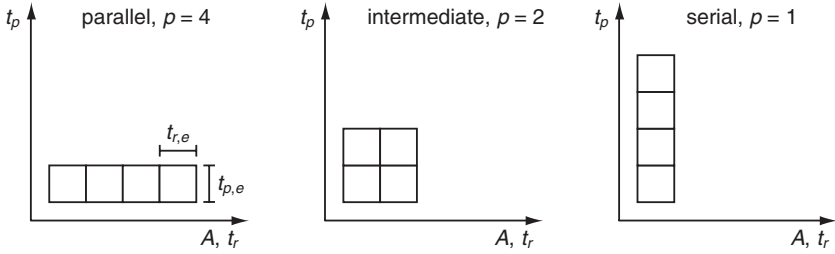
buffering is needed during reconfiguration. For the latter case we can calculate the buffer size  $B$  depending on reconfiguration time  $t_r$  and the application data throughput  $\phi_{app}$ :

$$B = \phi_{app} \cdot t_r = \frac{\phi_{app}}{\phi_{config}} \cdot \Psi. \quad (6.10)$$

Table 6.6 outlines the buffer size for several receiver functions and a range of reconfiguration times. We can observe that the data rate is reduced through all stages of the receiver. Hence, a reconfiguration-during-call scenario becomes easier to implement toward the end of the receiver chain. Obviously, the buffer size also increases with the bandwidth of the communication standard and the duration of the reconfiguration time.

A buffer can be implemented with on-chip or off-chip resources. Most modern FPGAs provide fast, embedded RAM blocks that can be used to implement first in–first out buffers. For example, Xilinx Virtex-5 FPGAs contain between 1 and 10 Mbit of RAM blocks. Larger buffers have to be realized with off-chip memories.

The performance of a run-time reconfigurable system is dictated by the reconfiguration downtime. If reconfigurable hardware is used as an accelerator for software functions, overall performance is usually improved despite the configuration overhead. In our case, we use reconfiguration to support multiple hardware functions in order to improve flexibility and reduce area requirements. In this case, the reconfigurable version of a design will have a performance penalty over a design that does not use reconfiguration. The reconfiguration of hardware usually takes much longer than a context switch on a processor. This is due to the relatively large amount of configuration data that need to be loaded into the device. The efficiency  $I$  of a reconfigurable design compared to a static design can be expressed as:



**Figure 6.14** Different spatial and temporal mappings of an algorithm with  $s = 4$  steps.

$$I = \frac{t_{static}}{t_{reconf}} = \frac{n \cdot t_p}{n \cdot t_p + t_r} = \frac{n}{n + \frac{t_r}{t_p}}. \quad (6.11)$$

The reconfigurable system becomes more efficient by processing more data between configurations and by improving the ratio of configuration time to processing time. We propose a more detailed analysis where we consider the effect of parallelism on processing time and configuration time. Many applications can be scaled between a small and slow serial implementation, and a large and fast parallel or pipelined implementation. FIR filter, AES encryption, or CORDIC (COordinate Rotation DIgital Computer) are examples of such algorithms.

Figure 6.14 illustrates the different spatial and temporal mappings of an algorithm with regard to processing time, area, and reconfiguration time. The processing time per datum  $t_p$  is inversely proportional to the degree of parallelism  $P$ . It can be calculated based on  $t_{p,e}$ , the basic processing time of one processing element,  $s$ , the number of steps or iterations in the algorithm, and  $P$ , the degree of parallelism:

$$t_p = \frac{t_{p,e} \cdot s}{P}. \quad (6.12)$$

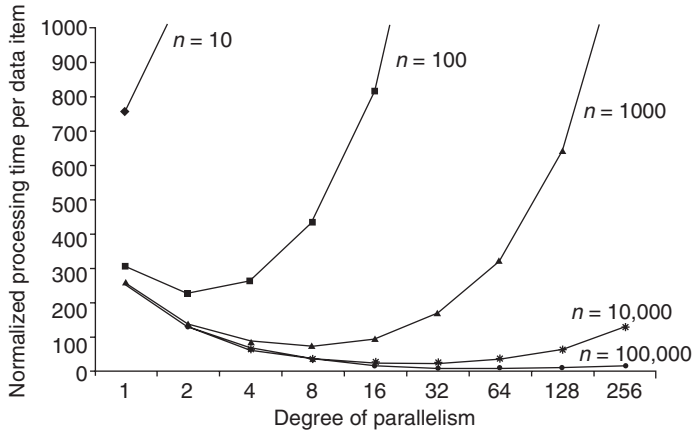
Parallelism speeds up the processing of data but slows down reconfiguration. This is because a parallel implementation is larger than a sequential one, and the reconfiguration time is directly proportional to the area as shown in Equation 6.9. The reconfiguration time  $t_r$  is directly proportional to the degree of parallelism  $P$ , where  $t_{r,e}$  is the basic reconfiguration time for one processing element:

$$t_r = t_{r,e} \cdot P. \quad (6.13)$$

We can now calculate the total processing time for a workload of  $n$  data items:

$$t_{total} = n \cdot t_p + t_r = \frac{t_{p,e} \cdot s \cdot n}{P} + t_{r,e} \cdot P. \quad (6.14)$$





**Figure 6.15** Normalized processing times for a range of workload sizes  $n$  and different levels of parallelism  $p$ . The number of steps  $s$  is set to 256 and we assume  $t_{r,e} = 5000t_{p,e}$ .

Figure 6.15 illustrates how parallelism can affect the optimality of the processing time. We consider an algorithm with  $s = 256$  steps, which is inspired by the observation that filters can have orders of 200 or higher. The plots are normalized to processing time per datum and we assume that the reconfiguration time  $t_{r,e}$  of one processing element is 5000 times the processing time  $t_{p,e}$  of one processing element. This value can vary depending on the application and target device but we estimate that at least the order of magnitude is realistic for current devices. We can observe that fully sequential implementations are beneficial for small workloads. In this case, the short configuration time outweighs the longer processing time. However, the overall time is still high due to the large influence of the configuration time. Large workloads benefit from a fully parallel implementation since the processing time is more dominant than reconfiguration time. In case of medium workloads, the degree of parallelism can be tuned to optimize the processing time.

In order to find the optimal degree of parallelism, we calculate the partial derivative of the function given in Equation 6.14 with respect to  $P$ :

$$\frac{\partial t_{total}}{\partial p} = \frac{t_{p,e} \cdot s \cdot n}{p^2} + t_{r,e}. \quad (6.15)$$

To find the minimum, we set Equation 6.15 to 0 and solve for  $P$ :

$$p_{opt} = \sqrt{\frac{s \cdot n \cdot t_{p,e}}{t_{r,e}}}. \quad (6.16)$$

The result  $p_{opt}$  is usually a real number, which is not a feasible value to specify parallelism. In order to determine a practical value for  $P$ ,  $p_{opt}$  can be interpreted according to Table 6.7.

**TABLE 6.7 Interpretation of  $p_{opt}$  to Determine a Practical Value for  $p$** 

|                      |                                                                       |
|----------------------|-----------------------------------------------------------------------|
| $0 < p_{opt} \leq 1$ | Fully serial implementation, $p = 1$                                  |
| $1 < p_{opt} < s$    | Choose $P$ such that $s/P \in \mathbf{Z}$ and $ p_{opt} - p $ minimal |
| $S \leq p_{opt}$     | Fully parallel implementation, $p = s$                                |

After determining the optimal degree of parallelism that reduces the overall processing time per workload and hence maximizes performance, it is still necessary to check if the implementation meets the throughput requirements of the application  $\Phi_{app}$ :

$$\frac{n}{t(p)_{total}} = \Phi_{hw} \geq \Phi_{app}. \quad (6.17)$$

The resulting area requirement  $A$  also has to be feasible within the total available area  $A_{max}$ . In summary, to implement an optimized design according to our model, the following steps have to be carried out:

1. Derive  $\Phi_{app}$ ,  $s$  and  $n$  from application.
2. Obtain  $\Phi_{config}$  for target technology.
3. Develop one design and determine  $t_p$  and  $A$ .
4. Calculate  $t_r$ ,  $t_{p,e}$ , and  $t_{r,e}$  using Equations 6.9, 6.12, and 6.13.
5. Find  $p_{opt}$  from Equation 6.16 and find a feasible value according to Table 6.7.
6. Calculate  $t_{total}$  using Equation 6.14 and verify throughput using Equation 6.17.
7. Implement design with  $P$  from step 5 and verify if its actual throughput satisfies the requirement.
8. Calculate buffer size  $B$  using Equation 6.10 and check  $A \leq A_{max}$ .

The above methodology can be adopted for a wide variety of applications and target technologies; it will find the highest performing version of the design. In order to find the smallest design that satisfies a given throughput requirement, one can try smaller values for  $P$  while checking Equation 6.17.

This approach can also be extended to address energy efficiency for reconfigurable designs [38]; a reconfigurable FIR filter is shown to be up to 49% more energy efficient and up to 87% more area efficient than a nonreconfigurable design.

## 6.10 CONCLUSIONS

Customization techniques can be applied in various ways to ASIC and to configurable technologies. We provide an overview of such techniques and

show how instance-specific designs and custom instruction processors can exploit customizability.

As technology advances, two effects become increasingly prominent:

1. integrated-circuit mask costs grow rapidly, making ASIC less affordable,
2. complexity of SOC design and verification keeps rising.

The technologies discussed in this chapter address these issues directly: various degrees of prefabrication customization reduce both the need for ASIC technology and the design complexity. Reconfigurable technologies such as FPGAs offer significant flexibility in the form of postfabrication customization, at the expense of overheads in speed, area, and power consumption.

Customization and configurability, in addition to their widespread adoption in commercial systems, also constitute exciting research areas, with recent progress in SOC design reuse [215], synthesizable datapath fabric [253], dynamically extensible microprocessors [48], customizable multiprocessors [90], and many others. Moreover, dynamically reconfigurable processors are beginning to be adopted commercially, such as the D-Fabrix from Panasonic, DRP-1 from NEC Electronics, and FE-GA from Hitachi [17]. It is also reported [74] that ARM processor and interconnect technologies, including ARM cell libraries and AMBA interconnect technology, would be adopted and optimized for Xilinx FPGA architectures. There is little doubt that customization and configurability will continue to play an important part in electronic systems for many years to come.

## 6.11 PROBLEM SET

1. Provide examples for several application domains that would benefit from postfabrication customization.
2. Some reconfigurable devices support pipelined interconnects. What are the pros and cons of pipelined interconnects?
3. Some FPGA companies provide a way of producing a structured ASIC implementation of an FPGA design, effectively removing the reconfigurability. Why do they do that?
4. Early FPGAs contain just a homogeneous array of fine-grained logic cells, while more recent ones are more heterogeneous; in addition to the fine-grained cells, they also contain configurable memory blocks, multiplier arrays, and even processor cores. Explain this evolution of FPGA architectures.
5. A subset of the instructions for a machine  $M$  can be accelerated by  $n$  times using a coprocessor  $C$ .
  - (a) A program  $P$  is compiled into instructions of  $M$  such that a fraction  $k$  belongs to this subset. What is the overall speedup that can be achieved using  $C$  with  $M$ ?

- (b) The coprocessor  $C$  in part (a) above costs  $j$  times as much as  $M$ . Calculate the minimum fraction of instructions for a program that  $C$  has to accelerate, so that the combined system of  $M$  and  $C$  is  $j$  times faster than  $M$ .
  - (c) The performance of  $M$  is improving by  $m$  times per month. How many months will pass before  $M$  alone, without the coprocessor  $C$ , can execute the program  $P$  in part (a) as fast as the current combined system of  $M$  and  $C$ ?
6. Explain how Equation 6.7 can be generalized to cover  $m$  custom instructions.
7. A database search engine makes use of run-time reconfiguration of the hash functions to reduce the amount of processing resources. The search engine contains  $P$  processors operating in parallel; each processor can be reconfigured to implement one of  $h$  hash functions. The total number of words,  $w$ , in the input dataset is divided into  $\ell$  subsets of words; each subset is processed using a particular hash function with one bit per word used to indicate whether a match has occurred. The indicator bit is stored along with the corresponding word in temporary memory, and such temporary data are processed by the next hash function in the processor after reconfiguration. The match indicator bit is updated in each iteration and the process continues until the data have been processed by all  $h$  hash functions. Let  $T_h$  denote the critical path delay of the hash function processor, and  $T_r$  is the time for reconfiguring the processor to support a different hash function. It takes  $m$  cycles to access the memory, and the average number of characters per word is  $c$ . Consider the worst case that all the hash functions are required all the time—the analysis will become more complex if it is possible to abort the matching process if a match does not occur.
- (a) How long does it take to process one subset of data?
  - (b) How long does it take to process all the data?
  - (c) Given that each character contains  $b$  bits, how many bits are required for the temporary storage?
8. To assess the effect of reconfiguration overheads on energy efficiency, consider developing an analytical model in the same spirit as the one in Section 6.9.2, involving an application with:
- $n$ , the number of packets or data items processed between two successive reconfigurations,
  - $s$ , the number of processing steps in the algorithm.
- A reconfigurable implementation is characterized by the following parameters:
- $A$ , the area requirement of the implementation,
  - $P$ , the amount of parallelism in the implementation,
  - $t_p$ , the processing time for one packet or datum,

- $t_r$ , the reconfiguration time,
- $P_p$ , the power consumed during processing,
- $P_c$ , the computation power which is a component of  $P_p$ ,
- $P_o$ , the power overhead which is a component of  $P_p$ ,
- $P_r$ , the power consumed during reconfiguration.

The reconfigurable device is characterized by:

- $\phi_{config}$ , the data throughput of the configuration interface,
- $\Theta$ , the configuration size per resource or unit of area.

Recall that energy is the product of power consumption and the associated time duration.

Given power consumption for computation,  $P_c$ , is directly proportional to  $P$ , the degree of parallelism, and there is a constant power consumption overhead,  $P_o$ , and a constant power consumption for reconfiguration,  $P_r$ :

- What is the computation energy  $E_c$  for processing  $n$  data items?
- What is the energy overhead  $E_o$  due to  $P_o$ ?
- What is the energy for reconfiguration  $E_r$ , given that the reconfiguration time is directly proportional to  $P$ ?
- What is the total energy per data item involved in computation and reconfiguration?
- Find the optimal degree of parallelism that minimizes the energy per datum.